

Master Thesis

University of Europe for Applied Sciences

Major: Data Science

Topic:

Prompt Framing Sensitivity in Large Language Models for Code Generation and Reasoning Tasks

Author: Duc Toan Huynh

Matriculation Number: 78784310

First supervisor: Prof. Dr. Iftikhar Ahmed

Second supervisor: Dr. Sami ur Rahman

Submitted on: August 14, 2026

Declaration of Authorship

Name: Duc Toan Huynh Matriculation No.: 78784310

Title of the thesis: Prompt Framing Sensitivity in Large Language Models for Code and Reasoning Tasks

With my signature, I confirm to be the sole author of the thesis presented. Where the work of others has been consulted, this is duly acknowledged in the thesis' bibliography. All verbatim or referential use of the sources named in the bibliography has been specifically indicated in the text.

The thesis at hand has not been presented to another examination board. It has not been part of an assignment over my course of studies and has not been published. The paper version of this thesis is identical to the digital version handed in.

Berlin, August 14, 2026

. . . . . .

(Signature: Duc Toan Huynh)

Declaration on the Use of Generative Artificial Intelligence (AI) Systems

Name: Duc Toan Huynh Matriculation No.: 78784310

Title of the exam: Prompt Framing Sensitivity in Large Language Models for Code and Reasoning Tasks

I have used the following artificial intelligence (AI)-based tools in the creation of my work:

1. OpenCode — code assistance for the experiment scripts and analysis notebooks
2. ZCode — thesis structure, drafting, and editing support
3. Grammarly — spelling and grammar correction

I further declare that

- ☒ I have actively informed myself about the performance and limitations of the above-mentioned AI tools,
- ☒ I have marked the passages taken from the above-mentioned AI tools,
- ☒ I have checked that the content generated with the help of the above-mentioned AI tools and adopted by me is actually accurate,
- ☒ I am aware that, as the author of this work, I am responsible for the information and statements made in it.

I have used the AI-based tools mentioned above as shown in the table below.

AI-based support tool	Usage	Parts of the work affected	Remarks
OpenCode	code assistance	experiment scripts and notebooks	verified
ZCode	thesis structure, drafting, editing support	all chapters	reviewed by the author
Grammarly	spelling and grammar correction	all chapters	—

Berlin, August 14, 2026

. . . . . .

(Signature: Duc Toan Huynh)

Abstract

Large Language Models (LLMs) are increasingly used in coding assistants and agentic workflows, where small prompt-framing choices affect parser compatibility, verbosity, latency, and correctness. However, common advice about tone, role assignment, and formats like JSON or XML remains under-tested across models and tasks.

This thesis examines prompt-framing effects through a full-factorial experiment with 18 prompt variants (3 tones $\times$ 2 roles $\times$ 3 formats). With 50 base tasks per task kind (Python code generation and mathematical reasoning), we conducted 1,800 runs per model across 6 local and 2 cloud LLMs, resulting in 14,400 runs in total. We measure Pass@1 correctness, duration, completion tokens, and output compliance.

Results show that model ability dominates correctness, while prompt effects are smaller, task-specific, and model-specific. Nemotron and DeepSeek strongly skew pooled averages. A mixed-effects logistic analysis with base-task random intercepts confirms the main contrasts. Prompt framing affects behaviors more than correctness: polite prompts increase output length, expert role framing slightly hurts cloud model performance, and JSON reduces the math pass rate for local models. In contrast, structured formats (JSON/XML) improve pass rates in API Python code generation.

The study found no universal prompting technique that reliably improves model performance. A strategy that helps one model may hurt another. Therefore, prompt engineers should test policies on their target models, instead of using universal prompt advice. Reliable LLM-based coding systems need explicit output constraints, robust parsers, validation loops, and local prompt-policy testing.

Keywords: Large Language Models, Prompt Engineering, Code Generation, Mathematical Reasoning, Empirical Software Engineering

Contents

Abstract iv

List of Figures viii

List of Tables ix

List of Abbreviations xi

1 Introduction 1

1.1 Motivation . . . 1
1.2 Objective . . . 2
1.3 Thesis Scope . . . 2
1.4 Purpose and Goal . . . 2
1.5 Thesis Outline . . . 3

2 Theoretical Background and Related Work 4

2.1 Prompt Sensitivity and Prompt Engineering in LLMs . . . 4
2.2 LLMs for Code Generation and Software-Engineering Tasks . . . 5
2.3 Structured Prompts, Output Constraints, and Parser Compatibility . 6
2.4 Evaluation of Reasoning and Final-Answer Extraction . . . 7
2.5 Research Gap . . . 8

3 Experimental Design 10

3.1 Research Questions . . . 10
3.2 Factorial Prompt Design . . . 10
3.3 Models Evaluated . . . 12
3.4 Tasks and Datasets . . . 14
3.5 Execution Environment and Run Protocol . . . 15
3.6 Outcome Measures . . . 17

4 Statistical and Analytical Strategy 19

4.1 Primary Contrasts . . . 19
4.2 Stratification by Task Kind and Model Group . . . 19

4.3 Mixed-Effects Models as the Main Inferential Basis 20
4.4 Multiple Comparisons Correction 21
4.5 Mediation and Behavioral Controls 22
4.6 Dominant-Model Sensitivity 22

5 Results and Empirical Analysis 24
5.1 Data Integrity and Run Coverage 25
5.2 Model Ability Dominates Correctness 25
5.3 Prompt-Factor Effects on Code Tasks 28
5.4 Prompt-Factor Effects on Math Tasks 29
5.5 Mixed-Effects Inferential Results 31
5.6 Output Behavior: Tokens, Duration, and Compliance 32
5.7 Mediation and Behavioral-Control Results 35
5.8 Clustered Linear-Probability Robustness Check 37
5.9 Dominant-Model Sensitivity and Aggregation Risk 38
5.10 Error and Failure Modes 40

6 Discussion and Threats to Validity 42
6.1 Main Interpretation 42
6.2 Implications for LLM-Based Coding Systems 42
6.3 Why Code and Math Differ 43
6.4 Reconsidering Prompt-Engineering Folklore 43
6.5 Practical Prompt-Policy Recommendations 44
6.6 Threats to Validity 45
6.6.1 Internal Validity 45
6.6.2 Construct Validity 45
6.6.3 Statistical Conclusion Validity 46
6.6.4 External Validity 47
6.6.5 Evaluation Validity and Audit Evidence 48

7 Conclusion and Outlook 49
7.1 Summary of Contributions 49
7.2 Data Availability and Replication Package 50
7.3 Statements and Declarations 50
7.3.1 Funding Statement 50
7.3.2 Competing Interests 50
7.3.3 AI Tools Declaration 50
7.3.4 Ethics Statement 51
7.4 Outlook and Future Work 51

List of terms **52**

Bibliography **53**

Annex **63**

List of Figures

3.1 Experimental pipeline from task sources through the prompt matrix, model execution, extraction/evaluation, and analysis. 12

5.1 Model pass rates by task kind. The figure highlights that between-model variation is larger than the main prompt-factor effects reported in later sections. 26

5.2 Prompt-factor effects on output behavior for local code and local math. Panel (a): effects on completion tokens and duration, in percent. Panel (b): effects on result-only compliance, in percentage points. 33

5.3 Prompt-factor correctness effects before and after controlling for completion tokens, result-only compliance, and duration. 36

List of Tables

2.1 Comparison with the closest prior studies. ✓ = included, ∼ = partial or single-family, – = not included. Behav. = behavioral outcomes beyond accuracy (tokens, latency, compliance). 8

3.1 Prompt framing factors and levels. 10
3.2 Local Ollama models evaluated in the experiment. 13
3.3 Cloud API models evaluated in the experiment. 13
3.4 Run counts by model group and task kind. 15

5.1 Key findings at a glance. Correctness effects are mixed-effects average marginal effects in percentage points; behavior effects are percent changes unless marked pp. 24
5.2 Data-integrity and run-coverage checks. 25
5.3 Model-level pass rates and output behavior. 27
5.4 Local code prompt-factor contrasts. 28
5.5 API Python prompt-format contrasts. 29
5.6 Local math prompt-factor contrasts. 29
5.7 API math role-framing contrast. 30
5.8 Mixed-effects logistic results for pass/fail by stratum: odds ratios and average marginal effects (AME) in percentage points, with 95% intervals and posterior p-values. 31
5.9 Local code behavior effects. 34
5.10 Local math behavior effects. 34
5.11 Local code correctness contrasts before and after adjustment for measured behavioral variables. 35
5.12 Local math correctness contrasts before and after adjustment for measured behavioral variables. 37
5.13 Dominant-model sensitivity of pooled pass rates. 38
5.14 Prompt-factor effects with and without the dominant model: clustered-LPM percentage-point contrasts and mixed-effects average marginal effects (AME pp). 38

5.15 Nemotron 3 Nano local math: task-level solve pattern across the 18 prompt conditions (50 base tasks). 40

List of Abbreviations

AI Artificial Intelligence.

AME Average Marginal Effect.

API Application Programming Interface.

CoT Chain of Thought.

FDR False Discovery Rate.

GLMM Generalized Linear Mixed Model.

GSM8K Grade School Math 8K.

IFEval Instruction-Following Evaluation.

JSON JavaScript Object Notation.

LLM Large Language Model.

LPM Linear Probability Model.

MBPP Mostly Basic Python Problems.

RQ Research Question.

SWE-bench Software Engineering Benchmark.

XML Extensible Markup Language.

1 Introduction

Large Language Models (LLMs) are widely used as assistants for software engineering work: planning, code generation, debugging, test generation/automation, documentation, and agentic tool workflows [1–5]. In those systems, models' responses are passed to a parser, executed by a test harness, extracted into a function body, checked by an automated judge, or used as input to another tool. Therefore, not only correctness but also the surface behavior matters. A response may contain the right algorithm but still fail because the harness cannot parse it: it may include extra markdown, explanations, invalid JSON/XML, no clear final answer, or text that the evaluation pipeline cannot easily extract.

1.1 Motivation

Those settings make prompt framing an important part of system design. Developers often use techniques such as polite language, assigning expert roles, pressuring for accuracy, or formatting the prompt in JSON or XML [6–8]. Such common advice may be useful in some contexts, but evidence of the effectiveness of many prompt rules is not strong. Prior work shows that prompt wording, formatting, and ordering can change model behavior in different directions [9–12]. A prompt that improves one model on one task may fail on another model or task. A prompt that helps a cloud model return parseable code may harm a local math model by encouraging compact answers. A role prompt may clarify the domain expectation for one model, but may make another model more verbose.

This led to two problems. The first problem is the reproducibility problem for empirical LLM research. If prompt phrasing changes model behavior in ways that are not reported or controlled, the resulting performance may be caused by prompt design rather than model ability [13]. The second problem is the reliability problem for coding agents. If prompt choices are selected from folklore rather than experiment and validation, an engineering workflow may become slower, more expensive, less parsable, or produce more errors.

1.2 Objective

This thesis studies prompt framing sensitivity: how semantically similar prompts lead to different behaviors due to framing choices. We examine three factors: tone (neutral, polite, aggressive), role (no role and expert role), and input format (plain-text, JSON, XML). These three factors create an 18-category prompt matrix, and each base task is evaluated under every category, across coding and mathematical reasoning tasks.

1.3 Thesis Scope

The main software-engineering task investigated in this thesis is Python code generation. The model is asked to generate Python code satisfying given requirements; an evaluator then extracts the code from the response and evaluates it against a list of tests. The secondary domain is mathematical reasoning, where only the final answer will be extracted and evaluated. The distinction matters: code workflows often benefit from clean, parseable output, whereas in math only the final number matters, even though deriving it may require enough space for intermediate reasoning.

The primary result of this research is intentionally conservative. While the capacity of the model overshadows task performance, the influence of prompting is still observed: it influences verbosity, latency, result-only compliance, and sometimes even Pass@1. However, the severity of this influence depends on the task type, model, and evaluator. A neutral or polite tone might not always work, and the same goes for expert roles and formats. Thus, the influence of prompting has to be determined in specific setups, including the model, task type, evaluator, parser, and latency constraints.

1.4 Purpose and Goal

This thesis is guided by four central questions regarding the size of prompt-framing effects, their impact on specific tasks and models, changes in observable behavior, and how relevant behavioral factors contribute to explaining differences in correctness. The detailed research questions are outlined in Section 3.1.

This thesis makes four primary contributions:

- **A systematic experiment**: Full-factorial prompt-framing experiment com-

bining tone, role framing, and prompt format across 14,400 LLM runs.

- **Evaluating beyond correctness alone**: A joint evaluation of both correctness and observable behaviors, tracking multiple outcomes such as response length, speed, and compliance alongside accuracy [14].
- **Model- and task-stratified analysis**: Reporting results separately for each model and task category, showing that prompt effects are smaller than model effects but still meaningful for output behavior and engineering reliability.
- **Mediation and sensitivity analysis**: Examining when prompt effects are associated with verbosity, compliance, and runtime, and when pooled averages are dominated by a single strong model.

1.5 Thesis Outline

This thesis is organized into seven chapters as follows:

Chapter 1 (Introduction) presents the motivation, research objectives, thesis scope, and primary contributions.

Chapter 2 (Theoretical Background and Related Work) surveys existing literature on prompt engineering, prompt sensitivity, LLM code generation, structured prompts, and reasoning evaluation.

Chapter 3 (Experimental Design) details the research questions, factorial prompt matrix, evaluated local and cloud models, task datasets, and execution environment.

Chapter 4 (Statistical and Analytical Strategy) describes the mixed-effects logistic regression models, linear probability models, multiple comparison corrections, and behavioral mediation framework.

Chapter 5 (Results and Empirical Analysis) presents empirical findings across model groups, correctness metrics, token usage, latency, compliance, and failure mode audits.

Chapter 6 (Discussion and Threats to Validity) synthesizes key insights, practical prompt-policy guidelines, and internal, construct, statistical conclusion, and external validity threats.

Chapter 7 (Conclusion and Outlook) summarizes the main findings, replication package, and outlines directions for future research.

2 Theoretical Background and Related Work

This chapter provides the background knowledge and literature context required to understand the prompt-sensitivity concepts and empirical methods applied in this thesis.

2.1 Prompt Sensitivity and Prompt Engineering in LLMs

Since large language models with strong few-shot and instruction-following abilities appeared in 2020, prompt engineering has become an essential part of working with these models [1]. Recently, prompting techniques have evolved from simple task descriptions to include instructions, examples, input/output format, role descriptions, reasoning cues, and conversational style. Recent surveys describe prompt engineering not as a collection of standard rules but as a broad and heterogeneous design space with many proposed techniques [15, 16]. Systematic reviews in educational contexts further examine how prompt engineering practices are being integrated into curricula [17]. A growing line of automatic prompt-optimization work instead treats prompts as searchable or compilable artifacts [18–20].

Early research on prompt-programming and few-shot calibration work also showed that prompts are not merely presentation layers but can significantly influence model behaviors. Reynolds and McDonell describe prompts as natural-language programs that communicate task intention and structure [21]. Zhao et al. demonstrated that prompt format, example choice, and example order can create a large variance in few-shot accuracy [9]. Lu et al. found that prompt example permutations alone can change performance from near-random to near state-of-the-art [10]. This is consistent with findings on in-context learning as implicit, context-dependent reasoning, as shown in [22]. Subsequent work has extended these findings to superficial prompt features. Indeed, Sclar et al. found that minor changes in format can be responsible for significant differences in performance, such as spacing or delimiter choice [11]. Zhu et al. studied model robustness to adversarial perturbations of prompts [23],

while Zhuo et al. analyzed the effect of prompt formats on code-generation evaluation [12]. Finally, even automatically discovered prompts that read as nonsense to humans can be better than manually created prompts, as reported in [24]. All these findings give reason to treat prompts as elements of the experimental design, rather than as incidental text.

Nevertheless, most of the studies in prompt-sensitivity examine primarily the aspect of correctness. While correctness is an important and necessary aspect in a production software application, it is insufficient. There can be correct answers that take too long or are overly complex for quick interpretation. This supports one of the ideas in behavioral testing of NLP systems, which is that other characteristics besides correctness are also important [25]. Accordingly, one of the points made in this thesis is that correctness has to be distinguished from the output behavior measured with the help of certain indicators such as token count, time of completion, result-only compliance, empty responses, and extraction failures.

The social context and tone are also taken into consideration in this area of study. Research on emotional stimuli suggests that emotional discourse may enhance the quality of generated messages [6], whereas research on politeness shows that it influences the form and content of the messages being generated [7]. Another element from the studies on sycophancy is that the output of bots can reflect the praise, agreement, or pressure to comply that the user exhibits [26]. The phenomenon of role-playing has also been studied as a way to influence the style and content of communication [27]. Current research is focused on aligning the output of LLMs with the user's needs through various techniques of prompting and fine-tuning [28]. These findings suggest that tone should be taken into account in this area of study, but they do not provide general principles. In other words, a polite prompt can produce different outcomes in different contexts.

2.2 LLMs for Code Generation and Software-Engineering Tasks

Code generation is the main software engineering application of LLMs. Benchmarks such as HumanEval and MBPP evaluate whether generated Python programs satisfy hidden or visible tests [3, 29]. For more realistic scenarios, SWE-bench extends evaluation to the repository level [4], and subsequent benchmarks broaden the picture to diverse function calls and complex instructions [30] as well as to many programming languages [31]. Recent open-source code models such as CodeLlama, StarCoder

2, WizardCoder, and Magicoder have substantially advanced the state of the art in code generation capabilities [32–35]. Comprehensive surveys of code generation methods and challenges provide additional context for understanding the current state of the art [36]. Those evaluations measure function correctness objectively rather than subjective preference. They also reveal a practical problem in evaluation: they depend on the process of extracting and executing code from model responses; rigorous re-evaluations show that execution-based accuracy is often lower than headline numbers suggest [37]. Therefore, generated code must be well recognizable, syntactically correct, and compatible with the test suites.

For these reasons, code generation is particularly suitable for studying prompt framing. In programming workflows, useful products are often not explanations, but functions, patches, commands, or structured outputs. Prompt framing affects whether the model returns clean code, code wrapped with markdown, text with code, or malformed output. A prompt can therefore change the pass rate by changing parser compatibility, even when the model's ability to solve the task is unchanged.

This thesis uses small, controlled Python tasks rather than repository-level tasks: task difficulty stays constant while framing changes, so prompt effects can be isolated. Chapter 3 discusses what this design does and does not represent.

2.3 Structured Prompts, Output Constraints, and Parser Compatibility

In LLM applications, it is common to use structured prompts and output constraints. According to [8], developers use JSON, XML, schemas, field names, or strict response instructions when framing prompts. Although structured output improves the compatibility of model outputs with software systems, it does not guarantee positive results. Merely placing inputs inside XML/JSON formats does not ensure that the model will produce valid structured output, and overly complicated formatting may result in the model losing helpful contextual information. A different technique suggests using a more complex form of grammar-based generation at the decoding stage [38], but this thesis intentionally excludes this method from the analysis to isolate the prompt-side framing approach.

In this work, plaintext, XML, and JSON formats are tested as prompt framing methods. The purpose of this analysis is not just to identify whether a structured format brings about positive results, but also whether it changes the behavior and

correctness of the model, and whether such changes depend strongly on the model and task being used (Python code vs. math).

This difference is important because users often confuse input wrapping with output control. Users may provide their tasks in the form of JSON while asking the model to create code or final numbers. In this regard, wrapping is a kind of input structure rather than ensuring a certain output form. In the paper, this problem is dealt with by monitoring both the correctness (pass rate) and the structural compliance (e.g., result-only or code-only) of the answers.

In the field of software engineering, this difference influences the way engineers construct pipelines rather than simply their terminology. Using JSON/XML to wrap prompts improves readability but doesn't guarantee the response format of the model. The model may still generate text, markdown, code, or incorrect structures. So, downstream applications require both input formatting restrictions and output validation processes. This thesis classifies the wrapping formats as a subcomponent of the entire model-tool interface.

2.4 Evaluation of Reasoning and Final-Answer Extraction

Benchmarks in mathematical reasoning such as GSM8K evaluate whether models can find correct answers to word problems [39], whereas MATH features competition-level questions [40]. Studies on reasoning techniques such as zero-shot reasoning and chain-of-thought have shown that prompts affect the flow of multi-step reasoning [41, 42]; this work has been extended in self-consistency sampling [43], least-to-most decomposition [44], and search-based reasoning [45]. Recent reinforcement learning models, such as DeepSeek-R1, demonstrate how the use of this technique can improve multi-step problem solving [46]. Process-supervision studies also show that evaluating intermediate steps rather than just final answers leads to better results in mathematics [47, 48]. Recent studies concerning reasoning techniques have looked at what these models can achieve regarding problem complexity [49]. However, it still takes considerable time to evaluate answers to problems, as it is necessary to perform extraction, normalization, and comparison. Furthermore, reasoning traces are unlikely to correspond to what the model has actually computed [50].

In this thesis, math problems serve as a secondary validation domain with a different

output contract: code tasks reward executable artifacts and parser compatibility, whereas math tasks require deriving a correct answer even when the final output must be compact.

The mathematical perspective helps to differentiate two concepts that are easily confused. Instructions requiring only the final answer may lead to easier evaluation; however, this does not imply that deriving the solution is simpler. Reasoning visible in the output does not indicate that the internal thought process behind it is faithful. Thus, this research stipulates that longer responses are not automatically considered better, and shorter ones are not automatically considered worse. The main conclusions show the relationships observed between the prompt format, the length of the response, the level of compliance, and the accuracy of the answer.

2.5 Research Gap

Table 2.1 illustrates how this master thesis relates to previous work within pertinent dimensions. Three principal observations can be made. First, existing studies in the area mainly consider isolated variables when examining the issue of prompt sensitivity. For instance, Sclar et al. [11] analyze the impact of surface formatting, Lu et al. [10] work with the order of examples, etc. Nevertheless, it is worth noting that no study has considered tone, role, and wrapper format together. Therefore, their possible interactions remain unexplored. Second, on the topic of prompt sensitivity, previous works have also explored the importance of code generation in studies [12]; however, their findings have not been incorporated into the context of reasoning and assessment results other than accuracy, nor into mediation-style analyses of the observed differences. Third, broad multi-metric evaluations like HELM [14] measure behaviors beyond accuracy but do not isolate prompt framing as the experimental variable. Furthermore, the most relevant tone studies report only accuracy, leaving the cost, latency, and compliance aspects of prompt framing unquantified.

Table 2.1: Comparison with the closest prior studies. ✓ = included, ∼ = partial or single-family, – = not included. Behav. = behavioral outcomes beyond accuracy (tokens, latency, compliance).

Study	Multi	Tone	Role	Fmt	Code	Reas.	Behav.	Fact.	Repro.
Zhao et al. [9]	∼	–	–	✓	–	–	–	–	–
Lu et al. [10]	∼	–	–	∼	–	–	–	–	∼
Sclar et al. [11]	✓	–	–	✓	–	–	–	∼	✓
Zhu et al. [23]	✓	–	–	∼	–	–	–	–	✓
Li et al. [6]	✓	∼	–	–	–	∼	–	–	∼

Study	Multi	Tone	Role	Fmt	Code	Reas.	Behav.	Fact.	Repro.
Yin et al. [7]	✓	✓	–	–	–	∼	–	∼	∼
Zhuo et al. [12]	✓	–	–	✓	✓	–	–	∼	∼
Liang et al. [14]	✓	–	–	–	✓	✓	✓	–	✓
This thesis	✓	✓	✓	✓	✓	✓	✓	✓	✓

Beyond these studies, this thesis also contributes: (i) a comprehensive factorial assessment of tone, role framing, and format (18 variations), thus measuring each factor against a stable baseline instead of in isolation; (ii) an analysis of 8 models ranging from local Ollama deployment systems to hosted APIs, treated as complementary settings rather than pooled benchmarks; (iii) an evaluation of executable code as well as math verification for the final answer under the same prompt matrix; (iv) behavioral outcomes (tokens, time, result-only compliance, and failure modes) reported together with pass@1 and a mediation-style decomposition in associational terms; and (v) mixed-effects inferential analysis, dominant-model sensitivity checks, and public replication materials. The approach is useful in terms of understanding how prompt framing functions, but not as a general mechanism for ensuring reliability.

3 Experimental Design

This chapter presents the methodology and experimental framework used to evaluate prompt framing sensitivity.

3.1 Research Questions

In this study, we investigate four central research questions:

- **RQ1**: How large are prompt-framing effects relative to model effects for code-generation and math-reasoning tasks?
- **RQ2**: Do tone, role framing, and prompt-wrapper format affect correctness, and are effects consistent across task kinds and model families?
- **RQ3**: How do prompt factors affect observable behavior such as completion tokens, duration, result-only compliance, empty responses, and extraction failures?
- **RQ4**: To what extent are correctness effects explained by behavioral variables such as verbosity, result-only compliance, and duration?

3.2 Factorial Prompt Design

This thesis adopts a $3 \times 2 \times 3$ factorial design with three tones (neutral, polite, aggressive), two role-framing levels (no role, expert/developer role), and three prompt formats (plain text, JSON, XML), creating 18 prompt variants for each base task. The prompt framing factors and levels are summarized in Table 3.1.

Table 3.1: Prompt framing factors and levels.

Factor	Levels	Purpose
Tone	Neutral, Polite, Aggressive	Test social framing and pressure cues

Factor	Levels	Purpose
Role Framing	No Role, Expert/Developer Role	Test role/domain framing
Wrapper Format	Plain Text, JSON, XML	Test structured prompt wrapping

Task content remains the same across variants, only framing changes. The `Format` column in the prompt matrix determines how the task is formatted (plaintext, JSON, XML). It is therefore a prompt format, not a guaranteed structured output. Full prompt templates are treated as replication documentation, rather than being fully posted in the main content.

Below is a compact prompt skeleton to illustrate the intervention. Within each task kind, the same base problem text is inserted into the `task` slot for each of 18 variants. The variable cells change the instruction phrasing via tone and role framing, and also change whether the same instruction/task pair is represented as plaintext, JSON, or XML. Output-only constraints for the API runs are applied separately via system prompts, not as fields in the prompt matrix.

```
Plaintext:
{role/background sentence}
{neutral, polite, or aggressive task instruction}
Task: {PYTHON_PROBLEM or MATH_PROBLEM}

JSON:
{
 "instruction": "{role/background sentence + task instruction}",
 "task": "{PYTHON_PROBLEM or MATH_PROBLEM}"
}

XML:
<prompt>
 <instruction>{role/background sentence + task instruction}</instruction>
 <task>{PYTHON_PROBLEM or MATH_PROBLEM}</task>
</prompt>
```

The experimental factors were intentionally designed to reflect normal user behavior rather than adversarial attacks. The polite prompts use grateful wording, the aggressive prompts apply pressure, the role prompts assign realistic personas, and

the structured prompts wrap the task in JSON or XML. Consequently, the current research uses standard prompt engineering methods rather than red-team perturbation methods. Since every baseline task occurs in all variants, each attribute comparison evaluates output for the same underlying task distribution.

The crossed design also helps address an issue inherent to prompt comparison experiments: concurrently altering multiple factors. In manual prompt tuning, developers frequently change tone, amount of detail, prompt length, and format simultaneously. While such a prompt may lead to practical results, it is difficult to determine which component of the prompt was responsible for the effect. By separating tone, roles, and the wrapper format, our approach enables measuring each of the elements against a stable baseline while preserving realistic prompts.

Figure 3.1 summarizes the end-to-end experimental pipeline from task sources through prompt construction, model execution, extraction, evaluation, and analysis.

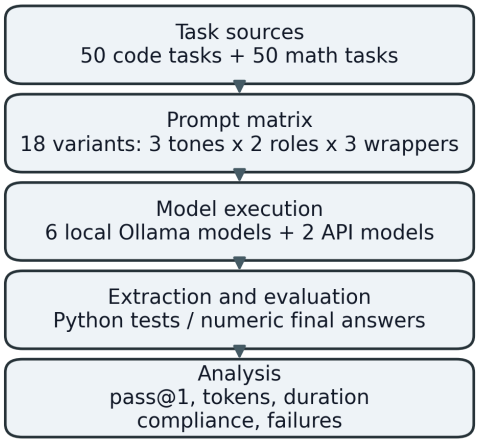


Figure 3.1: Experimental pipeline from task sources through the prompt matrix, model execution, extraction/evaluation, and analysis.

3.3 Models Evaluated

The experiment includes two model groups: local models and cloud models (or API models). The six Ollama-hosted local models and their specifications are detailed in Table 3.2:

Table 3.2: Local Ollama models evaluated in the experiment.

Model	Parameters	Ollama package size
nemotron-3-nano-4b	3.97B	2.64 GiB
deepseek-coder-6.7b	6.74B	3.57 GiB
qwen2.5-7b	7.0B	4.36 GiB
mistral-7b-instruct	7.24B	4.07 GiB
llama3.1-8b	8.0B	4.58 GiB
gemma2-9b	9.0B	5.07 GiB

The local package sizes are quantized Ollama package sizes, not FP16 memory requirements. Quantized local deployment on consumer hardware is enabled by memory-efficient inference techniques [51]. Model identifiers and public model-card information are used for the local model descriptions where available [52–57].

The cloud API models evaluated in this experiment are listed in Table 3.3:

Table 3.3: Cloud API models evaluated in the experiment.

Reporting label	API model identifier
Gemini Flash	`gemini-3.1-flash-lite-preview`
DeepSeek V4 Flash	`deepseek-ai/DeepSeek-V4-Flash`

Local and API models are reported separately because they differ in deployment settings, latency, response style, training and post-training procedures, serving infrastructure, and overall capability. Because these differences are confounded with the deployment regime itself, performance gaps between the two groups cannot be attributed to model ability alone.

The models were selected intentionally heterogeneous. The six local models represent lightweight, open-source deployments that a practitioner could run through Ollama, while the two API models represent commercial systems with stronger infrastructure and different serving behavior. This split is not to rank all available LLMs or create a general benchmark. It is used to test whether the effects of prompt-factor patterns remain stable when moving between different local and cloud settings. As the results show, stability cannot be assumed.

Separating local and API results also protects the validity of latency and output behavior. Local runtimes are governed by local inference stacks and quantized weights, whereas API runtimes are influenced by hosted serving infrastructure and network

conditions. Combining them into a single timing metric would be misleading. Therefore, this thesis treats local and API findings as distinct but complementary evidence of how prompt sensitivity reacts to different deployment regimes.

3.4 Tasks and Datasets

The study evaluates two task types: Python code generation and final-answer mathematical reasoning. Code generation acts as the primary software-engineering benchmark, requiring models to produce functional Python code to meet specific requirements. An automated evaluation pipeline then extracts the generated code and executes it against test cases. Mathematical tasks function as a secondary validation domain, where the evaluator isolates the model's final answer and compares it against the expected result.

Local and API models were evaluated on different problem sets. Local code tasks are derived from high-school graduation and mock-exam math problems from the ToanMath THPT archive and converted into executable Python code [58, 59], while local math tasks use single-answer numeric problems filtered from VietJack's 2026 grade-10 entrance exam archive [59, 60]. API models use a paired MathNet-derived competition-math set with 50 problems for math and their Python conversions for code [59]. Because the models, task datasets, serving environments, and execution settings all differ between the two groups, the local and API experiments are designed as complementary experiments under different settings, not as head-to-head comparisons on a shared problem source; throughout this thesis, differences between the two groups are interpreted as differences between experimental settings rather than as model-ability rankings.

For every combination of model and task type, 50 base tasks are evaluated across 18 prompt variations, producing 900 individual runs per model–task kind. In total, the experiment encompasses 14,400 runs, 10,800 from local models and 3,600 from API models. The total run counts broken down by model group and task kind are presented in Table 3.4.

The task sets are intentionally created with an equal number of prompts and levels of difficulty; otherwise, the comparison could create the false impression that one prompt type is better than the other on easier tasks. The research design guarantees that the framing effect can be examined for each task without reflecting only the problem-related content.

Both domains serve different functions. Software development is the first domain, as it is centered around code generation that produces usable products. Math problems are used for validation. The problems in this domain require multiple derivations but only produce a short answer that can be judged as objectively correct. Therefore, the combination of the two domains allows us to determine whether the prompt framing strategies that enhance code generation will have a similar effect on the final answer to a math problem.

Because coding challenges involve converting mathematical problems into Python, their representativeness for software-engineering code generation requires explicit discussion. This controlled task design was chosen deliberately for three reasons. First, math-derived problems have definitive, automatically checkable answers and test suites, allowing pass/fail scoring without human judgment across all 14,400 runs. Second, the short, self-contained nature of these problems keeps task difficulty constant while only the prompt framing changes, which is necessary to isolate tone, role, and format effects in a factorial design. Third, a single-attempt protocol on compact tasks ensures uniformity across the evaluation pipeline, including extraction, execution, and test harnesses. What this design primarily measures is controlled initial code generation: how prompt framing affects a model's first, single-shot attempt and its compatibility with an automated evaluator. However, it does not represent repository-level software development, such as multi-file context, iterative debugging, test-repair loops, code review, or long-horizon agent behavior. Therefore, the conclusions of this thesis should be interpreted as evidence regarding prompt framing in controlled initial code generation and final-answer reasoning, rather than as claims about LLM performance in realistic software projects.

Table 3.4: Run counts by model group and task kind.

Group	Models	Task kinds	Runs per model-task kind	Total runs
Local models	6	2	900	10,800
API models	2	2	900	3,600
Total	8	2	-	14,400

3.5 Execution Environment and Run Protocol

We evaluate each prompt variant using a single-response approach, with pass@1 as the main metric. We keep the base task consistent across all variants, meaning that each variant is evaluated on every base task. Furthermore, to keep the factorial

design fully intact, we retain blank outputs and errors rather than discarding them.

The local result files cover runs from 12 to 14 May 2026, logging Ollama model identifiers, timestamps, execution duration, prompt and completion token counts, generated responses, extracted code/numerical values, and evaluation results. Similarly, the API result files log runs conducted during the same period, detailing model identifiers and labels, timestamps, raw usage statistics, API response metadata, text, and evaluation results.

The evaluation framework uses an OpenAI-compatible chat-completions interface for its API runners. The default API settings in the runner are a temperature of 0, a maximum of 2048 generated tokens, a 120-second API timeout, a 10-second timeout for Python code execution, and a 2-second delay between requests. System prompts strictly instruct the model to "Output only the code, no explanations" for Python programming tasks and "Output only the final result, no explanations" for mathematical tasks. Furthermore, the system retains quota error logs and token-rotation metadata when utilizing multiple API keys, and holds execution states so it can resume running later if interrupted.

For local evaluations, the Ollama runners capture response times and token usage directly from Ollama's metadata. The specific configuration for the local math runner includes a temperature of 0, a 300-second request timeout, a 256-token generation limit, and a 0.2-second delay between calls. Legacy local code notebooks follow the same automated extraction and testing protocols, maintaining the 10-second execution timeout per test. Across all configurations, the top-p parameter is left at its default value for the respective provider or Ollama, meaning it is not treated as an experimental variable. Finally, the replication package includes comprehensive documentation: sanitized API configurations, details on the local Ollama setup, notebook environment specifications, and the retry and error-handling mechanisms applied to the six local models.

The system prompts have been made intentionally strict to enable automated evaluation and reduce ambiguity of results. However, it should be emphasized that automated evaluation still requires compliance measurement. Models may nevertheless provide additional explanatory text, ignore the required output format, generate no output at all, or produce unexpected formats. Although there is no need to get rid of all such examples, it is crucial that they demonstrate how the prompt policy works in practical applications as opposed to being just experimental noise.

The single-response approach is very important for interpretation purposes. It keeps the above calculations valid and prohibits iterative fixing of the generated responses.

Actual agents use feedback to correct their mistakes; however, it should be noted that if revisions are allowed, it will not be easy to extract the effect of the prompt framing. Evaluating the model against only one attempt allows separating the effect of the manipulated prompt parameters on the model-evaluator pair.

3.6 Outcome Measures

The experiment results are presented in the form of pass@1. For coding tasks, for a task to be considered passed, the produced Python code must pass all tests, including two examples provided in the prompts and five hidden tests. For mathematics problems, the solution passes if the extracted answer is the same as the required one after normalization.

Secondary outcomes describe observable behavior, including instruction-following-style compliance with the expected output form [61]:

- **Completion tokens**: number of generated tokens.
- **Duration**: response latency in seconds.
- **Result-only compliance**: whether the response is close to the expected compact output form.
- **Code-only compliance**: whether code responses are mostly clean, executable code.
- **Empty responses**: responses with no usable output.
- **Extraction failures and error modes**: cases where the output cannot be parsed or evaluated.

These outcomes are central to this thesis because prompt design can affect a system's usefulness even without improving correctness.

We interpret these behavioral outcomes as engineering signals: completion tokens approximate cost and verbosity, duration estimates latency, and compliance approximates the ease with which downstream tools can consume the response. Furthermore, empty outputs and extraction failures expose brittle points in the evaluation pipeline. Rigorous execution-based evaluation of code LLMs shows that evaluator design choices—extraction rules, test harnesses, and normalization—are themselves a significant source of variation across runners [37]. The broader shift toward using LLMs as automated judges introduces additional considerations about evaluation reliability and bias [62]. Reporting these dimensions alongside pass@1 enables us to distinguish prompts that yield genuine accuracy gains from those that simply trade

off accuracy for cost, cleanliness, speed, or parser compatibility.

Our definition of compliance is practical rather than philosophical. Result-only compliance does not show that the model understood the instruction, and code-only compliance does not show that the code is correct. Instead, the evaluation only considers whether the response fulfilled the formal requirements of the evaluation pipeline, since LLMs function as parts of larger integrated systems.

4 Statistical and Analytical Strategy

This chapter details the analytical methodology, statistical models, and hypothesis testing strategies employed to evaluate experimental results.

4.1 Primary Contrasts

To analyze prompt factors, we compare each experimental condition against a standard baseline: neutral tone, no role, and plain text. Correctness differences are expressed in percentage points of the success rates, whereas behavioral effects are expressed in the number of tokens used and response duration. The results are supplemented by p-values; however, they are considered exploratory unless adjusted for multiple hypothesis testing.

The use of percentage-point differences is advantageous since they provide an easily understandable basis for prompt-engineering decisions. For instance, a shift in the pass rate of four points corresponds to four additional or fewer successes out of 100 trials, without having to account for cost and latency. It simplifies the engineering heuristic compared to odds ratios. For this reason, we focus on the effect sizes and their directions, and we regard p-values merely as measures of statistical uncertainty rather than binary thresholds for significance.

4.2 Stratification by Task Kind and Model Group

We have separated our results according to task domain (code vs. math) and deployment method (local vs. API). This stratification is critical because our tests show that the effect of the prompt varies depending on both the task type and the model used. Using data from just one cumulative measure can cause us to miss important variations. For example, JSON and XML formats improve performance on API-enabled Python tasks; conversely, for mathematical tasks run locally, the use of JSON formats decreases accuracy.

In addition, this stratification provides protection from overgeneralization. A pooled estimate may be statistically precise but risks leading to misguided conclusions if it averages together different mechanisms. Code generation relies on executable artifacts and parser compatibility, while mathematical reasoning depends on answer extraction and numerical evaluation. Local and API models differ in their default verbosity, instruction adherence, latency, and serving infrastructure. Rather than treating these variations as statistical noise to be averaged out, our analysis considers them integral components of the phenomenon under investigation.

4.3 Mixed-Effects Models as the Main Inferential Basis

As each base task is assessed multiple times according to all 18 prompt types and under each model in the stratum, the pass/fail observations are not independent. Mixed-effects logistic regression is the appropriate inferential framework for such clustered benchmark data [63]. Therefore, the primary analysis in this thesis is a mixed-effects logistic regression for pass/fail status carried out separately for local code, local math, API Python, and API math. Each model includes fixed effects for the five contrasts of prompt factors (aggressive and polite vs. neutral tone, role framing vs. no role, and JSON and XML vs. plaintext), fixed effects for models in the stratum, and a random intercept per base task with a single shared variance component, which allows modeling the repetition of the same 50 base tasks explicitly instead of treating them as independent rows. Model estimation is done using variational Bayes as implemented in statsmodels' `BinomialBayesMixedGLM`. We provide the results in the form of odds ratios with 95% intervals and average marginal effects (AME) in percentage points, averaged over the observed covariates and the estimated task random-intercept distribution, with intervals from posterior draws.

Similarly, the repeated-measures structure is applied to the behavioral outcomes. Log completion tokens and log duration are modeled with linear mixed-effects models using the same fixed-effect structure and a base-task random intercept. Result-only compliance, being binary, is modeled with the same logistic mixed-effects specification as pass/fail. However, for API code, compliance remains constant under the system prompts, precluding model estimation; therefore, these results are reported descriptively.

The percentage-point contrasts and linear probability models (LPMs) with model-

level and base-task fixed effects are retained as a descriptive and interpretable layer: a 4 percentage-point increase means 4 more (or fewer) successful trials out of 100, which is easier to handle from an engineering perspective than odds ratios. Furthermore, linear probability models with standard errors clustered by the base task are kept as a robustness check against mixed-effects estimates. We note that predicted probabilities from linear probability models are not bounded by the unit interval (i.e., they can fall outside the 0 to 1 range); therefore, clustered LPMs are not seen as formal estimates but rather as sensitivity measures, while logistic regression remains the main method of inference for pass/fail claims. Robustness expectation is directional rather than exact. P-values for the mixed model, the clustered LPM, and raw contrasts will not necessarily be identical, but the sign and approximate magnitude of the headline effects should agree. Where they do not, we investigate and report the discrepancy rather than presenting an isolated significant result as a headline finding.

4.4 Multiple Comparisons Correction

Given the many different comparisons made in this experiment, both in terms of factors and models, p-values must be interpreted carefully. We report uncorrected p-values on the main contrast tables for readability; subsequently, we apply the Benjamini-Hochberg false discovery rate correction to evaluate model-specific local baseline contrasts. Findings are considered provisional if they fall outside the corrected list of model-specific baselines, such as the impacts of API math role framing.

The goal of the correction approach is intentionally targeted. This thesis makes no claims or assumptions about all possible pairs of comparisons being confirmatory. Instead, it separates headline aggregate contrasts, corrected local model-based baseline contrasts, and exploratory details to ensure that the story does not turn isolated low p-values into broad prompt rules. Findings that survive correction or have been determined to be directionally consistent across robustness testing will receive greater weight than isolated marginal contrasts.

This distinction shapes the reporting language throughout the thesis. A finding can have interesting practical significance even when it is not treated as confirmatory, especially if it highlights a possible implementation issue such as higher token costs or lower compliance. On the other hand, a finding that is statistically significant but has a small effect size may carry limited importance in engineering. The interpretation of findings is therefore dependent on effect size, uncertainty, robustness,

and operational relevance.

4.5 Mediation and Behavioral Controls

The mediation-style analysis uses behavioral decomposition rather than proving the existence of hidden reasoning processes. Log completion tokens, result-only compliance, and log duration are the adjustment variables. The goal is to describe how much of each observed prompt contrast remains after adjustment for observable behavior related to cost, latency, and parser compatibility.

The mediators used in this analysis (tokens, compliance, and duration) are all post-treatment variables that themselves respond to the prompt framing. Because these variables change in response to the treatment, adjusting for them cannot isolate a clean causal direct effect, and we do not interpret the adjusted coefficients as proving a mechanism. The analysis is nonetheless useful for system design when stated in associational terms. If a pass-rate contrast is attenuated after adjustment for measured behavioral variables, those behaviors are associated with part of the observed contrast, a pattern consistent with a parser-compatibility or verbosity pathway. Conversely, if a contrast remains after adjustment, the measured behaviors account for only part of it, and the remainder reflects unmeasured aspects of the model–evaluator interaction that this study cannot identify. In both cases, the interpretation relies on observable outputs rather than hidden reasoning processes.

4.6 Dominant-Model Sensitivity

The dominant-model analysis comprises a two-level robustness check. First, it examines how the pooled pass rates vary when the most powerful model within the group is excluded from the analysis. In the second and more critical step, it investigates whether the prompt effects themselves are distorted. This re-estimation takes place for each main tone, role, and format contrast, with and without the dominant model, relying on both the LPM percentage-point scale and the mixed-effects logistic scale. In this way, it becomes possible to assess the direction and magnitude of prompt effects across the panels.

In small model panels, this leave-one-dominant-model approach is particularly significant. When one model has an extraordinarily high pass rate, pooled prompt effects may simply reflect how that specific model responds to the prompts. Therefore,

reporting only the pooled effect of all models would create a misleadingly generalized result. The sensitivity analysis seeks to determine whether the same practical outcome still exists once this dominant model is extracted from the data. If it no longer exists, the thesis will directly state this dependence rather than smooth it out.

A structural limitation applies to the API group. Because there are only two cloud models, excluding DeepSeek V4 Flash leaves a single-model panel (comprising only Gemini Flash), meaning the leave-one-out estimates describe a single model rather than a model average. We report these estimates because they directly expose how much of each pooled API effect is attributable to the dominant model.

All of these individual analyses serve as the basis for the conservative evidentiary standard employed in this thesis. The strongest evidence is provided by patterns across tasks and model groups, similarity across summaries, and the impact on system behavior. Conversely, the weakest evidence is provided by a single p-value, pooled averages dominated by one model, or system behavior that does not align with correctness. This evidentiary standard reflects the goal of this thesis: to view prompt framing as an engineering control variable, rather than a universal benchmark.

5 Results and Empirical Analysis

Because of the varying stability of prompt effects across code/math and local/API settings, the results are organized by task type and model group. The local and API groups are complementary experiments run under different models, task datasets, and serving environments, so cross-group differences are reported descriptively and are not interpreted as head-to-head model comparisons. Model ability is generally more determinative of whether an answer is correct, while prompt framing is more determinative of observable behavior than of functional correctness.

This chapter is organized to answer the four research questions in turn. Section 5.1 verifies data integrity. Section 5.2 answers RQ1 by comparing model and prompt effect sizes. Sections 5.3–5.5 answer RQ2 on correctness, moving from descriptive contrasts to the main mixed-effects inference. Section 5.6 answers RQ3 on observable behavior. Section 5.7 answers RQ4 on behavior-adjusted contrasts. Sections 5.8–5.9 report robustness and aggregation risk, and Section 5.10 summarizes failure modes. Table 5.1 gives the key findings at a glance before the detailed subsections.

Table 5.1: Key findings at a glance. Correctness effects are mixed-effects average marginal effects in percentage points; behavior effects are percent changes unless marked pp.

Stratum	Correctness (pass/fail)	Output behavior	Details
Local code	Small role penalty (-1.0 pp); tone and format null	Polite tone: +12.4% tokens, +10.8% duration; JSON/XML improve compliance	Sec. 5.3, 5.5, 5.6
Local math	JSON penalty -4.1 pp, XML penalty -2.1 pp; role +2.1 pp and polite +1.6 pp gains	JSON: -19.8% tokens; polite tone: +11.0% tokens	Sec. 5.4, 5.5
API Python	JSON gain +3.4 pp, XML gain +5.2 pp	Wrappers reduce tokens and duration	Sec. 5.3, 5.5
API math	Role penalty -2.8 pp	Role framing: +83 tokens	Sec. 5.4, 5.5

Stratum	Correctness (pass/fail)	Output behavior	Details
Sensitivity	Local effects survive Nemotron removal; API Python effects attenuate without DeepSeek	–	Sec. 5.9

5.1 Data Integrity and Run Coverage

The Consolidated Results Package includes 12 local JSONL files and four API JSONL files. Each model-task file contains 900 records, and there are no duplicate full run keys. Each base task contains all 18 prompt variants for each model-task combination. No gaps were found in the prompt metadata join. Empty responses are rare: there are only 14 in the Local Results Package (all in the Nemotron code outputs) and zero in the API Results Package. The data-integrity and run-coverage verification results are detailed in Table 5.2.

Table 5.2: Data-integrity and run-coverage checks.

Check	Result
Local result files	12 JSONL files, 900 rows each
API result files	4 JSONL files, 900 rows each
Duplicate task keys	0
Prompt metadata join gaps	0
Empty local responses	14, all Nemotron code
Empty API responses	0

5.2 Model Ability Dominates Correctness

Model choice has a much larger effect than prompt-factor choice. Figure 5.1 shows that local code pass rates range from 0.4% to 22.8%, local math from 5.7% to 91.2%, API Python from 66.7% to 69.0%, and API math from 24.1% to 82.4%. The model-level pass rates, median durations, and completion token counts are presented in Table 5.3.

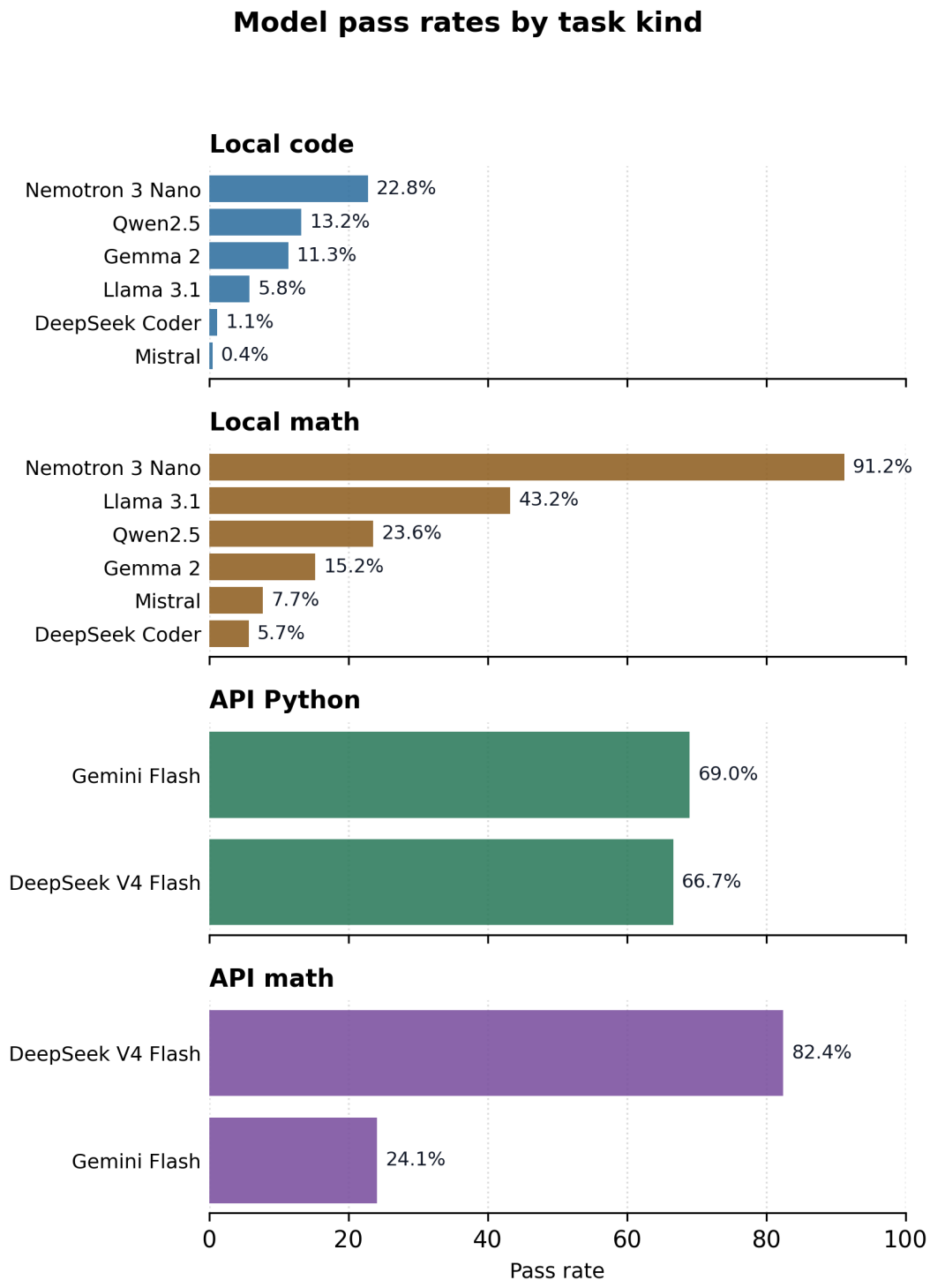


Figure 5.1: Model pass rates by task kind. The figure highlights that between-model variation is larger than the main prompt-factor effects reported in later sections.

Table 5.3: Model-level pass rates and output behavior.

Task kind	Model	Pass rate	Median duration	Mean completion tokens
local code	nemotron-3-nano-4b	22.8%	17.78s	1646
local code	qwen2.5-7b	13.2%	8.75s	316
local code	gemma2-9b	11.3%	5.52s	175
local code	llama3.1-8b	5.8%	11.16s	394
local code	deepseek-coder-6.7b	1.1%	9.04s	491
local code	mistral-7b-instruct	0.4%	9.12s	366
local math	nemotron-3-nano-4b	91.2%	3.15s	289
local math	llama3.1-8b	43.2%	5.09s	274
local math	qwen2.5-7b	23.6%	0.68s	6
local math	gemma2-9b	15.2%	0.80s	6
local math	mistral-7b-instruct	7.7%	0.89s	79
local math	deepseek-coder-6.7b	5.7%	5.35s	287
API Python	gemini-flash	69.0%	10.30s	359
API Python	deepseek-v4-flash	66.7%	16.12s	1352
API math	deepseek-v4-flash	82.4%	9.41s	1127
API math	gemini-flash	24.1%	3.40s	15

The differences between the various models are much greater than the differences between the prompt variants. For instance, the local math gap between Nemotron and DeepSeek Coder is 85.5 percentage points, and the greatest pooled local prompt format effect is 4.2 percentage points.

This establishes the scale for the rest of the results: prompt factors are secondary to much larger model and task differences, but still meaningful. A prompt-policy recommendation that ignores the actual model is misleading. The practical question is not whether better prompting can make a weak model perform like a strong one;

the evidence does not support that. The useful question is whether better framing makes the chosen model work more smoothly with the evaluator, cost less to run, and fail in fewer, more predictable ways.

5.3 Prompt-Factor Effects on Code Tasks

The pooled results for local code tasks show no significant tone or format effects on correctness. The pass rate for both neutral and aggressive prompts is 9.3%, while the pass rate for polite prompts is 8.8%. Role framing reduces the pass rate slightly (from 9.6% to 8.6%); the raw pooled contrast is not significant, but the mixed-effects and clustered models both support a small role penalty of about one percentage point. XML shows a slight advantage over plaintext and JSON, but the differences are small and not statistically significant. The prompt-factor contrasts for local code tasks are summarized in Table 5.4.

Table 5.4: Local code prompt-factor contrasts.

Factor	Contrast	Pass diff	p	Token diff	Duration diff
tone	aggressive vs neutral	+0.0 pp	1.000	+4	+0.10s
tone	polite vs neutral	-0.5 pp	0.601	+30	+0.96s
role	role vs none	-1.0 pp	0.185	+15	+0.27s
format	JSON vs plaintext	+0.2 pp	0.860	-16	-0.39s
format	XML vs plaintext	+0.8 pp	0.387	-21	-0.46s

For local code tasks, the results are more strongly behavior-based than correctness-based. Completion tokens and duration increase when polite prompts are used, but this does not impact the overall pass rate. Role prompts also increase verbosity. Using JSON/XML decreases the number of tokens and duration while achieving a higher level of result-only/code-only compliance. Ultimately, the behavioral gains driven by prompt usage do not result in meaningful improvements in the pooled pass rate for local code.

In API Python, JSON raises the raw pass rate by 3.2 percentage points while reducing token count by 105 and request time by 1.55 seconds. Similarly, XML raises the pass rate by 4.8 percentage points while reducing tokens by 77 and duration by 1.12 seconds. The raw two-proportion p-values fail to reach the 0.05 significance threshold, but the clustered fixed-effect p-values do. As a result, XML remains statistically significant, while JSON is positive but marginal. The prompt-format

contrasts for API Python tasks are shown in Table 5.5.

Table 5.5: API Python prompt-format contrasts.

Factor	Contrast	Pass diff	p	Token diff	Duration diff
format	JSON vs plaintext	+3.2 pp	0.244	-105	-1.55s
format	XML vs plaintext	+4.8 pp	0.074	-77	-1.12s

This comparison has important implications for engineering interpretation. Structured wrappers can assist coding workflows when they enhance parser compatibility and decrease unnecessary output, but this benefit does not apply universally across all model groups or task types.

The API Python result was the strongest positive finding for structured wrappers in this experiment, especially when considering the clustered robustness check. This result is also limited to two hosted models, one Python task family, and a very strict code-only system prompt. Consequently, it may be worthwhile to try using JSON or XML wrappers in the coding workflows of API-based development; this will be most beneficial when downstream tooling benefits from compact task boundaries. However, this study does not support applying the same wrapper policy across all model groups or task types.

5.4 Prompt-Factor Effects on Math Tasks

Analysis of the data shows that math tasks are more sensitive to prompt framing at the model-specific level than code tasks are. At the local level, using JSON reduces the pooled pass rate by 4.2 percentage points and decreases both tokens used and duration. Using XML reduces the pooled pass rate by 2.2 percentage points, though the unadjusted statistical contrast is not significant. Using role framing also has a positive, but marginal, effect on local math. The prompt-factor contrasts for local math tasks are listed in Table 5.6.

Table 5.6: Local math prompt-factor contrasts.

Factor	Contrast	Pass diff	p	Token diff	Duration diff
tone	aggressive vs neutral	+0.9 pp	0.539	+7	+0.29s
tone	polite vs neutral	+1.7 pp	0.280	+17	+0.55s
role	role vs none	+2.2 pp	0.083	+11	+0.29s
format	JSON vs plaintext	-4.2 pp	0.007	-32	-0.92s

Factor	Contrast	Pass diff	p	Token diff	Duration diff
format	XML vs plaintext	-2.2 pp	0.153	+1	-0.02s

Only some models are adversely affected by the local math format effect, as indicated by separate model analyses. Llama 3.1 dropped 15.7 percentage points under JSON and 9.7 percentage points under XML, while DeepSeek Coder dropped 4.3 percentage points under JSON and 5.7 percentage points under XML. The Qwen 2.5 model benefited from role framing, with an increase of 10.2 percentage points. The mixed results indicate that the ultimate conclusion is not that "JSON is bad"; rather, it is that "format effects are model- and task-dependent."

An analysis using the Benjamini-Hochberg correction procedure applied to the 60 local baseline contrasts supports a cautious interpretation of the results. Seven of the 60 have an uncorrected p-value < 0.05; however, after treating all 60 as one family, only two remain statistically significant: Llama 3.1's JSON penalty in math and Qwen 2.5's role-framing gain in math. When the code and math conditions are treated as two different families of 30 each, there are no remaining significant results for code, but there are 3 significant results for math: Llama 3.1 JSON, Qwen 2.5 role framing, and DeepSeek Coder XML. Therefore, the results of this analysis support the conclusion that sensitivity to prompt types is more of a phenomenon related to the math side of the dataset than the coding side.

For API math, role framing shows a small negative effect: it decreases the pass rate by 2.8 percentage points and increases tokens by 83. The raw comparison is not statistically significant; however, the clustered model check verified that the negative direction remains evident. The role-framing contrast for API math tasks is detailed in Table 5.7.

Table 5.7: API math role-framing contrast.

Factor	Contrast	Pass diff	p	Token diff	Duration diff
role	role vs none	-2.8 pp	0.238	+83	+0.47s

The combined math analyses answer RQ2 and RQ3 more robustly than pooling the individual math tables alone. The direction of format and role effects varies based on the model used and the task type. The math penalties observed with local JSON formatting serve as a cautionary reminder not to assume that structured wrappers will always improve performance. The Qwen role gain and the API role penalty confirm this for persona prompts; that is, framing a question with a role

can increase the pass rate, decrease it, or mainly change the verbosity of the output depending on the model and evaluator.

5.5 Mixed-Effects Inferential Results

Table 5.8 reports the main inferential analysis: mixed-effects logistic models for pass/fail including prompt-factor and model fixed effects and a random intercept per base task, fitted per stratum. The average marginal effects are expressed in percentage points so they can be read alongside the descriptive contrast tables above.

Table 5.8: Mixed-effects logistic results for pass/fail by stratum: odds ratios and average marginal effects (AME) in percentage points, with 95% intervals and posterior p-values.

Stratum	Contrast	Odds ratio [95% int.]	AME pp [95% int.]	p
API Python	Aggressive tone	1.05 [0.83, 1.34]	+0.6 [-2.3, +3.4]	0.686
API Python	Polite tone	1.05 [0.83, 1.34]	+0.6 [-2.2, +3.4]	0.686
API Python	Role framing	0.92 [0.76, 1.11]	-1.1 [-3.5, +1.2]	0.381
API Python	JSON wrapper	1.32 [1.04, 1.68]	+3.4 [+0.5, +6.2]	0.022
API Python	XML wrapper	1.55 [1.21, 1.97]	+5.2 [+2.4, +8.0]	<0.001
API math	Aggressive tone	1.05 [0.81, 1.38]	+0.5 [-2.1, +2.9]	0.697
API math	Polite tone	0.91 [0.70, 1.18]	-0.9 [-3.5, +1.5]	0.476
API math	Role framing	0.74 [0.60, 0.92]	-2.8 [-4.8, -0.8]	0.006
API math	JSON wrapper	0.87 [0.67, 1.14]	-1.3 [-3.8, +1.2]	0.325
API math	XML wrapper	0.78 [0.60, 1.02]	-2.3 [-4.7, +0.1]	0.069
Local code	Aggressive tone	0.98 [0.80, 1.20]	-0.1 [-1.0, +0.9]	0.858
Local code	Polite tone	0.89 [0.72, 1.09]	-0.5 [-1.4, +0.4]	0.265
Local code	Role framing	0.81 [0.68, 0.96]	-1.0 [-1.7, -0.2]	0.013
Local code	JSON wrapper	1.01 [0.83, 1.24]	+0.1 [-0.8, +1.0]	0.900
Local code	XML wrapper	1.15 [0.94, 1.41]	+0.7 [-0.2, +1.6]	0.164
Local math	Aggressive tone	1.11 [0.95, 1.29]	+0.9 [-0.5, +2.2]	0.198
Local math	Polite tone	1.20 [1.03, 1.40]	+1.6 [+0.3, +3.0]	0.019
Local math	Role framing	1.28 [1.13, 1.45]	+2.1 [+1.0, +3.2]	<0.001
Local math	JSON wrapper	0.62 [0.52, 0.72]	-4.1 [-5.4, -2.8]	<0.001
Local math	XML wrapper	0.78 [0.66, 0.91]	-2.1 [-3.4, -0.9]	0.001

According to the mixed-effects results, all of the headline findings are confirmed. The results of the main inferential analysis confirm the benefit of the structured format in API Python code for both JSON (OR 1.32, p = 0.022) and XML (OR

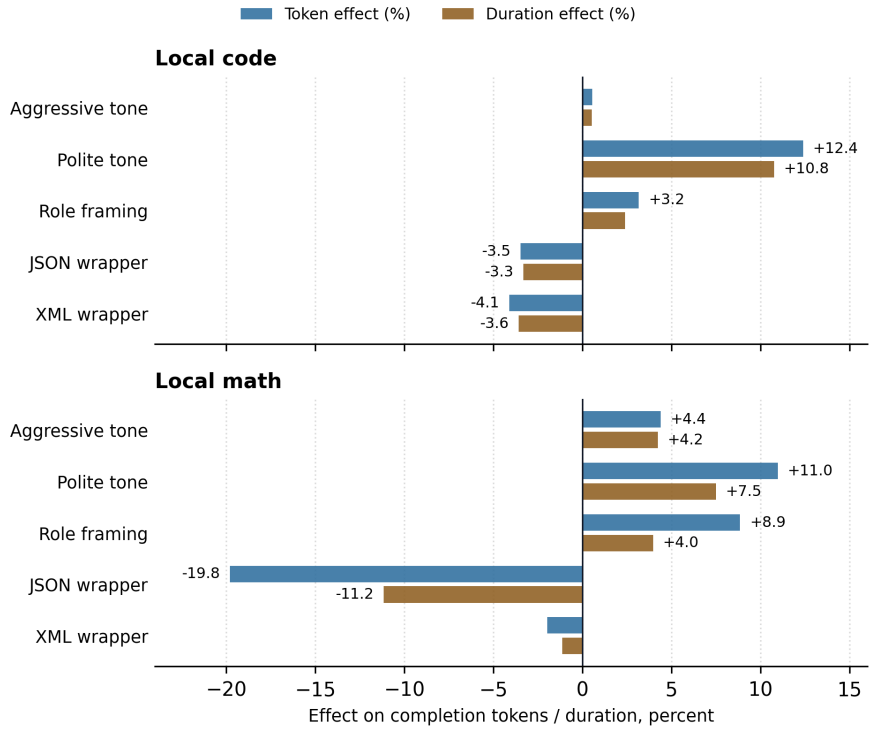
1.55, $p < 0.001$). The penalty for local math in JSON is clearly established (OR 0.62, $p < 0.001$); the penalty for local math in XML is also confirmed (OR 0.78, $p = 0.001$), as are the gain from local math in role-framing (OR 1.28, $p < 0.001$) and the small polite-tone gain in local math (OR 1.20, $p = 0.019$). The role-framing penalty in API math, which was regarded as exploratory in the descriptive analysis, was confirmed in the mixed model (OR 0.74, $p = 0.006$). The small penalty for the local-code role is also confirmed in the mixed analysis (OR 0.81, $p = 0.013$); however, at roughly one percentage point, it remains practically minor. No tone effects can be regarded as significant in either code stratum.

The behavioral mixed models present the same findings as the descriptive behavior tables. Using linear mixed models with random task-specific intercepts, polite language leads to an increase in log completion tokens and log duration in the local strata, while the use of both JSON and XML decreases these figures; additionally, role framing modestly increases them. These overall effect directions confirm what is seen in the descriptive behavior tables. Furthermore, the result-only compliance outcome remains constant for API code under strict system prompts, meaning no compliance model can be estimated in that context.

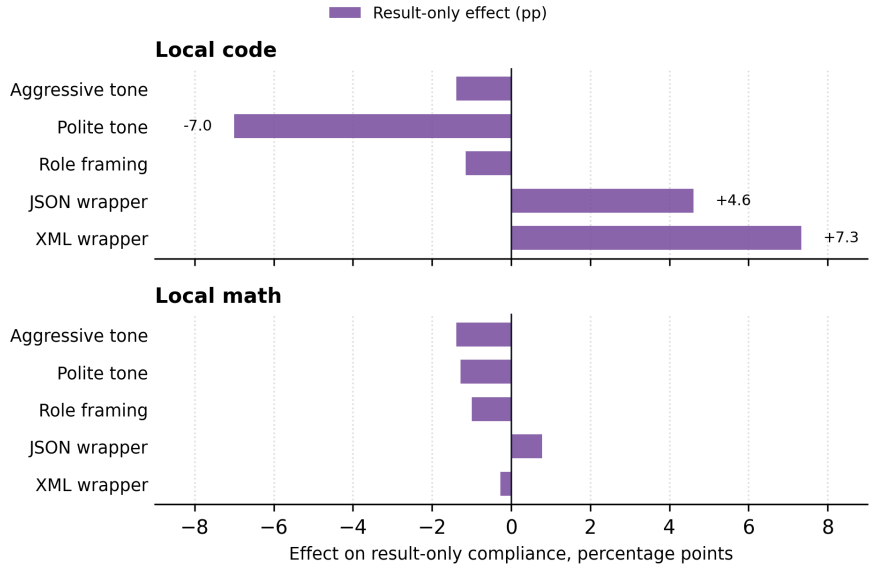
5.6 Output Behavior: Tokens, Duration, and Compliance

Prompt framing changes observable output behavior more reliably than correctness. In the case of coding tasks, a polite tone leads to 12.40% more completion tokens, a 7.00-percentage-point decline in result-only compliance, and an increase of 10.76% in duration. Role framing results in 3.17% more tokens and 2.41% longer duration. JSON and XML reduce tokens and duration but improve result-only compliance.

Figure 5.2 visualizes these behavior effects, split by unit: panel (a) shows token and duration effects in percent, panel (b) shows result-only compliance effects in percentage points. The largest behavior changes are not always correctness gains: polite tone increases output length and duration, while JSON strongly shortens local math responses. The observable output behavior effects for local code tasks are presented in Table 5.9, and for local math tasks in Table 5.10.



(a) Token and duration effects, percent



(b) Result-only compliance effects, percentage points

Figure 5.2: Prompt-factor effects on output behavior for local code and local math. Panel (a): effects on completion tokens and duration, in percent. Panel (b): effects on result-only compliance, in percentage points.

Table 5.9: Local code behavior effects.

Local code factor	Token effect	Result-only effect	Duration effect
Aggressive tone	+0.55%	-1.39 pp	+0.52%
Polite tone	+12.40%	-7.00 pp	+10.76%
Role framing	+3.17%	-1.15 pp	+2.41%
JSON wrapper	-3.46%	+4.61 pp	-3.32%
XML wrapper	-4.10%	+7.33 pp	-3.57%

In math tasks, polite prompts increase tokens and duration, and role prompts do the same, but JSON strongly reduces both. Result-only compliance is also relatively unchanged for math tasks because the majority of models have stable response patterns regardless of the prompt. Typically, Qwen, Gemma, and Nemotron provide short responses, whereas Llama and DeepSeek provide lengthy responses.

Table 5.10: Local math behavior effects.

Local math factor	Token effect	Result-only effect	Duration effect
Aggressive tone	+4.39%	-1.39 pp	+4.24%
Polite tone	+10.97%	-1.28 pp	+7.51%
Role framing	+8.86%	-1.00 pp	+3.98%
JSON wrapper	-19.80%	+0.78 pp	-11.16%
XML wrapper	-1.96%	-0.28 pp	-1.14%

Behavioral impacts are significant for applied systems. Even when the level of correctness doesn't change, a particular prompt may increase costs, latency, and parsing difficulty. On the other hand, a prompt can make the output easier for a parser to process, even if the model's actual performance on the task does not improve.

This is the clearest answer to RQ3. Prompt framing changes how models answer more reliably than it changes whether answers are correct. In coding tasks, this distinction is critical from an operations perspective, since excessive verbosity or a lack of clear delimiters in the response will prevent the answer from being automatically extracted. In mathematics tasks, this distinction manifests differently: many models already provide very short numerical outputs, leaving less room for behavioral

variations, though changes in token count or time will still signal different response strategies. Reporting both behavioral and correctness information in tandem will help prevent an efficiency gain from being confused with a quality gain.

5.7 Mediation and Behavioral-Control Results

The mediation-style analysis investigates how much of each prompt effect remains after adjustment for observable behavior. For code tasks, the descriptive polite-tone contrast is an overall 0.50 percentage-point decrease in pass rate. After adjustment for tokens, result-only compliance, and duration, the coefficient is attenuated and shifts to a positive 0.17 percentage points. In other words, the apparent polite-tone penalty in code generation is associated with the measured behavioral variables: polite prompts are associated with longer, slower, and less parseable outputs, and once those behaviors are adjusted for, no negative correctness contrast remains. Because the adjustment variables are post-treatment, this pattern is consistent with a verbosity/parser pathway but does not prove it.

The total prompt-factor correctness contrasts, and the coefficients after adjustment for measured behavioral variables are shown in Figure 5.3. The local code correctness contrasts before and after adjustment are listed in Table 5.11, and the corresponding local math contrasts are shown in Table 5.12.

Table 5.11: Local code correctness contrasts before and after adjustment for measured behavioral variables.

Local code factor	Total contrast	After behavioral adjustment
Aggressive tone	-0.00 pp	-0.01 pp
Polite tone	-0.50 pp	+0.17 pp
Role framing	-1.04 pp	-0.87 pp
JSON wrapper	+0.17 pp	+0.06 pp
XML wrapper	+0.83 pp	+0.82 pp

For math tasks, the JSON coefficient remains negative after adjustment. The total contrast is -4.17 percentage points; after adjustment for token amount, result-only compliance, and duration, the remaining coefficient is -2.84 percentage points. The measured behaviors are therefore associated with part of the observed contrast, but not all of it. This pattern indicates that JSON-style framing is associated with both the model's observable behavior and downstream correctness in this setting; it should not be interpreted as evidence about the model's hidden reasoning processes.

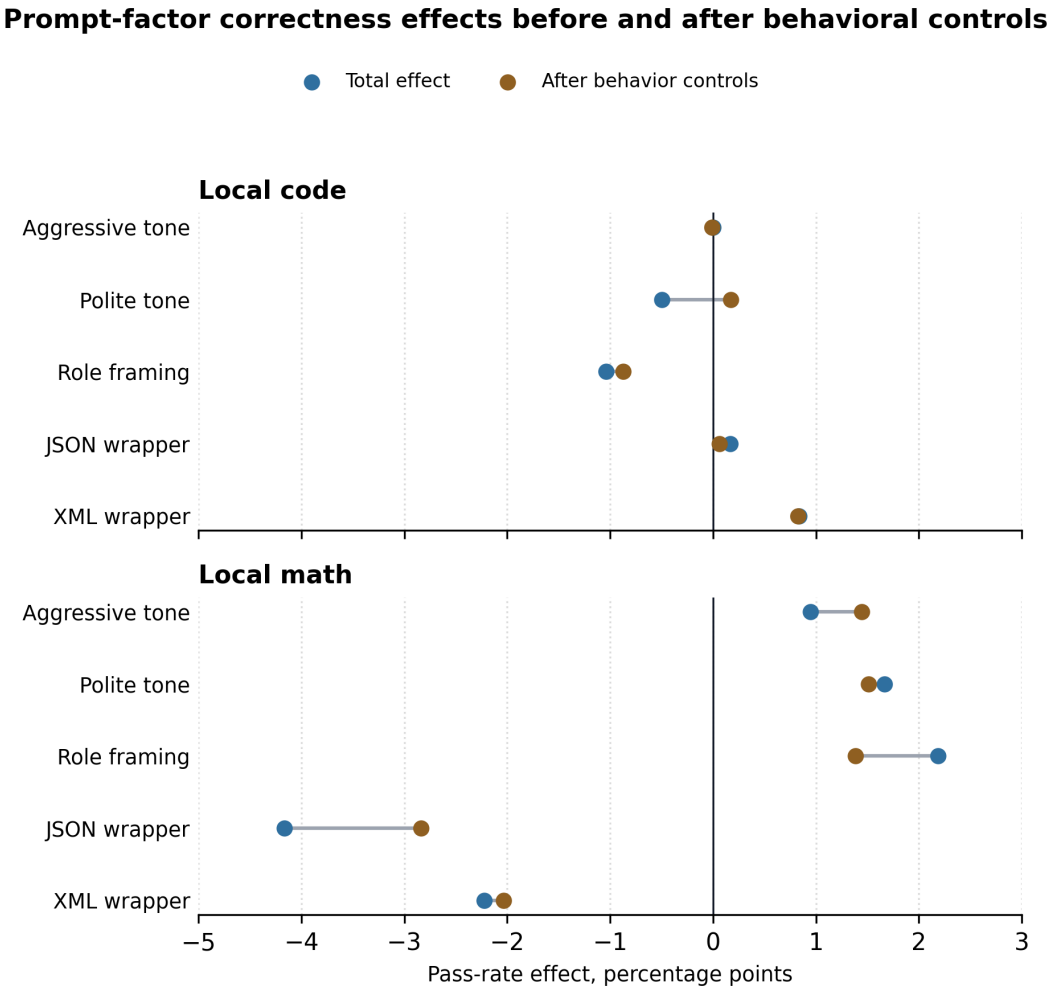


Figure 5.3: Prompt-factor correctness effects before and after controlling for completion tokens, result-only compliance, and duration.

Table 5.12: Local math correctness contrasts before and after adjustment for measured behavioral variables.

Local math factor	Total contrast	After behavioral adjustment
Aggressive tone	+0.94 pp	+1.45 pp
Polite tone	+1.67 pp	+1.51 pp
Role framing	+2.19 pp	+1.38 pp
JSON wrapper	-4.17 pp	-2.84 pp
XML wrapper	-2.22 pp	-2.04 pp

Thus, the mediation-style tables serve primarily as associational checkpoints and should not be viewed as a replacement for correctness results or as proof of a mechanism. For local code, the attenuated tone and role coefficients are consistent with a pipeline explanation in which tone and role framing mainly relate to verbosity and ease of parsing. For local math, the measured behavior variables account for only part of the JSON penalty. This residual negative coefficient should be viewed with caution; however, it is exactly the type of outcome that justifies validating prompt policies per task and model rather than trusting universal advice.

5.8 Clustered Linear-Probability Robustness Check

As a robustness check on the mixed-effects results, we fit pass/fail linear probability models separately by model group and task type, with prompt factors and model fixed effects and standard errors clustered by base task. The clustered LPM addresses the same repeated-task dependence as the mixed model with a different set of assumptions, and it does not deviate significantly from the main results while correcting for the repetition of the same 50 base tasks across all prompt variants and models.

In the clustered analysis, the core direction of headline effects remains stable and agrees with the mixed-effects model. For API Python, XML stays significant at +4.83 percentage points (clustered SE 1.77, p=0.006), while JSON is positive but marginal at +3.17 percentage points (clustered SE 1.73, p=0.068). For local math, JSON stands at -4.17 percentage points (clustered SE 1.11, $p < 0.001$) and XML remains negative at -2.22 percentage points (clustered SE 1.02, p=0.030). Local math role framing is positive at +2.19 percentage points (clustered SE 0.69, p=0.002), and polite tone is also positive at +1.67 percentage points (clustered SE 0.79, p=0.035).

The clustered check sharpens the key narrative instead of reversing it. For positive results, the most robust format is API Python XML, while the most robust negative format is local math JSON. Every effect that is significant in the mixed-effects model is directionally consistent in the clustered LPM, and the remaining effects are either small, model-dependent, or exploratory.

5.9 Dominant-Model Sensitivity and Aggregation Risk

A single dominant model can overshadow the pooled averages. Removing Nemotron leads to a drop in the local math pass rate from 31.1% to 19.1%. The API math pass rate drops by almost half, from 53.3% to 24.1%, when DeepSeek V4 Flash is removed. API Python is more stable because Gemini Flash has a similar Python pass rate to DeepSeek V4 Flash. The sensitivity of pooled pass rates to removing dominant models is detailed in Table 5.13.

Table 5.13: Dominant-model sensitivity of pooled pass rates.

Sensitivity dataset	Task kind	Full avg.	Without dominant
Local, no Nemotron	code	9.1%	6.4%
Local, no Nemotron	math	31.1%	19.1%
API, no DeepSeek	math	53.3%	24.1%
API, no DeepSeek	python	67.8%	69.0%

Pass-rate averages, however, do not show whether the prompt effects themselves are distorted by the dominant model. We therefore re-estimated every main tone, role, and format contrast with and without the dominant model in each group, using both the clustered LPM and the mixed-effects logistic model. Table 5.14 reports the percentage-point effects in both panels; for the API group, the leave-one-out panel contains only Gemini Flash, so those columns are single-model estimates.

Table 5.14: Prompt-factor effects with and without the dominant model: clustered-LPM percentage-point contrasts and mixed-effects average marginal effects (AME pp).

Stratum	Contrast	LPM full	LPM w/o dom.	AME full	AME w/o dom.
API Python	JSON wrapper	+3.2	+1.3	+3.4	+1.6
API Python	XML wrapper	+4.8	+2.7	+5.3	+3.1

Stratum	Contrast	LPM full	LPM w/o dom.	AME full	AME w/o dom.
API math	Role framing	-2.8	-2.4	-2.8	-4.1
API math	JSON wrapper	-1.3	-3.7	-1.3	-6.1
API math	XML wrapper	-2.3	-0.0	-2.4	-0.2
Local code	Role framing	-1.0	-0.9	-1.0	-0.9
Local math	Polite tone	+1.7	+2.3	+1.6	+2.3
Local math	Role framing	+2.2	+2.4	+2.1	+2.3
Local math	JSON wrapper	-4.2	-5.3	-4.1	-5.2
Local math	XML wrapper	-2.2	-2.5	-2.2	-2.4

The re-estimations indicate that the local prompt effects are not merely an artifact of Nemotron's dominance. In the absence of Nemotron, the local math JSON penalty has a slightly greater impact (-5.2 pp AME compared to -4.1 pp in the full panel) while still being highly statistically significant, and the two local math prompt gains (role framing and politeness) exhibit no observable change in their magnitude. Finally, even the small local-code role penalty is just as consistent between both estimates (-0.9 pp AME). Overall, while the leading local model shifts the pooled pass rate by 12 points, it leaves the prompt-effect estimates intact in both direction and magnitude.

The API group displays a different pattern, as expected in a two-model panel. The benefit of the API Python structured format shrinks with the removal of DeepSeek V4 Flash, causing the XML effect to fall from +5.3 to +3.1 pp AME and the JSON effect to decrease from +3.4 to +1.6 pp AME, and neither is shown to be statistically significant in a Gemini-only panel. Therefore, the combined API Python effect must be interpreted as a two-model average with the major input from DeepSeek, and not a feature attributable to each model. The API math role-framing penalty, on the contrary, still holds and becomes even greater in the Gemini-only panel (-4.1 pp AME), which means that it does not depend solely on DeepSeek. The API math XML penalty does not remain after the removal (-0.2 pp AME), thus emphasizing its exploratory character.

This sensitivity test lends support to the primary methodological point: that prompt-policy conclusions must not come from pooled averages only, but be derived from each model group and task type, with the dominant-model dependence of each effect stated explicitly.

Given the unusually high local math pass rate of 91.2%, Nemotron underwent an independent repository audit. The audited Nemotron math file consists of 900 dis-

tinct runs, balanced across 50 base tasks and 18 prompt variants, with zero duplicate task IDs. Since re-computing the pass/fail scores using `expected_answer` and `extracted_values` reproduced 821 out of 900 passes with zero mismatches against the stored `passed` field, and a scan found no expected-answer leakage, it is fair to say that the high score is stable across each prompt variant. This figure further demonstrates that the value in question can be treated as a real result, thus justifying the no-Nemotron sensitivity check.

To illuminate the 91.2% figure at the task level, we provide a breakdown of how the 50 local math tasks perform across the 18 different prompt conditions tested by Nemotron. As shown in Table 5.15, the high pass rate is not due to just a few lucky task variants; 42 of the 50 tasks pass in all 18 conditions, 6 tasks pass under some conditions, and only 2 tasks fail in all conditions. Thus, the high average reflects a model that consistently solves most of the task set across various framing conditions, rather than being a mere evaluation artifact affecting a small number of conditions.

Table 5.15: Nemotron 3 Nano local math: task-level solve pattern across the 18 prompt conditions (50 base tasks).

Solve pattern across 18 prompt conditions	Number of tasks (of 50)
Solved under all 18 conditions	42
Solved under some conditions (1–17)	6
Solved under no condition	2

5.10 Error and Failure Modes

The experiment shows some of the possible failure modes. Regarding code-based tasks, a few of these failures include wrong algorithms, mistakes in syntax, missing functions, or even outputs incorporating prose or markdown that the extractor cannot cleanly process. The use of both polite and role prompts may lead to parsing issues since they can lead to more explanatory text. Formats such as JSON and XML may make outputs more compact, but they do not guarantee correctness. The types of failures in math tasks include wrong answers, missing final-answer markers, too brief or vague answers, and extraction ambiguity. Using JSON reduces the duration and length of local math responses while decreasing the pass rate, suggesting that using compact formatting can lower accuracy in certain reasoning tasks even if the requested output is final-result-only. This does not mean that showing reasoning is always required; rather, the results show that less verbose responses

are associated with lower local math accuracy in this study. Representative cases demonstrate the contrast between task-related failures and parser or interface failures. In one Nemotron code run, the response returned by the model was empty, leading to the evaluation of `No code extracted`. In a different incident with Qwen 2.5, Python code was executed; however, the model used helper names that did not satisfy the requirements of the callable interface, due to which the code execution failed with name-resolution errors. One more instance occurred when a local math run was conducted with Qwen 2.5 using the JSON-format prompt. The result was that the answer `6` was given when the expected answer was `12`, showing the compact nature of the answer, which was easily extractable yet erroneous. These examples support the broader interpretation that correctness, extraction compatibility, and output behavior should be reported separately.

6 Discussion and Threats to Validity

This chapter synthesizes the empirical findings, discusses practical prompt-policy implications, and analyzes threats to validity.

6.1 Main Interpretation

The initial interpretation is balanced. Prompt framing is not a universal tool for accuracy, but it is indeed an engineering variable in action. When considering the LLM as a system, it is important to evaluate the prompt, not view it as the final word in LLM operations. The latest studies of multilingual LLMs indicate that prompt sensitivity differs depending on the language, which adds to the importance of the prompt parameters [64].

The effects of prompts are viewed more as interface effects between the model and the evaluator than as changes in model competence. Polite code prompts lead to an increase in explanatory language, making results harder to extract, while structured wrappers decrease verbosity. In local math tasks, the compactness introduced by some prompt types is associated with poorer quality of answers. The main takeaway is that prompts, parsers, validators, and task descriptions together create a measuring system. Recent research on reducing hallucinations in LLMs provides more methods of ensuring the reliability of results [65, 66].

6.2 Implications for LLM-Based Coding Systems

Reliable coding assistants need engineering beyond prompt design: explicit output contracts and automated validation. Workflows should separate reasoning from the final artifact, produce a clean code artifact at the end, and avoid relying on prompt tricks like politeness or role labeling for correctness. The debate continues about whether ChatGPT and similar tools are genuinely beneficial for software engineering or introduce new failure modes. The API Python results show the value of structured

wrappers in some coding workflows: raw pass-rate contrasts improve under JSON and XML while tokens and duration decrease, and the mixed-effects model supports both effects. This suggests that structured wrappers can improve compatibility with automated evaluation when they align with the task and model. The local math result shows the danger of turning this into universal advice.

The same principle applies to agentic workflows, where model outputs are consumed by downstream systems [67–69]. Recent work on tool learning with foundation models further demonstrates that LLMs must interface with external systems while maintaining output reliability [70]. A response that provides human-friendly explanations may be unusable for automated tools due to extra commentary or formatting inconsistencies, while machine-friendly output may be too terse for human debugging. Prompt policy must therefore be directed at the actual output consumer: the instruction should state the required artifact, the evaluator should accept common valid forms, and systems serving both human and automated consumers should separate these phases rather than asking one prompt to satisfy incompatible goals.

6.3 Why Code and Math Differ

Programming and mathematical evaluations impose different requirements for output results. Code evaluation favors artifacts that are executable and compatible with the parser. Extra explanations become barriers by interfering with extraction. Structured wrappers help to signify the nature of the required output and reduce irrelevant text.

Mathematical evaluation, on the other hand, requires a correct answer derived from the reasoning process. The output contract asks for just the answer; therefore, extraction is easier, although very compact framing may coincide with less successful answer derivation. In this experiment, JSON framing lowers the number of local math tokens and reduces generation duration, while the pass rate drops. This does not prove that visible reasoning is required; it only shows that format constraints interact with task types in ways that affect downstream accuracy.

6.4 Reconsidering Prompt-Engineering Folklore

The results offer a mixed perspective on prompt folklore. The principle of "being polite" as a prompt technique was found to provide no improvement in correctness;

in fact, it resulted in larger token counts and increased response times without any benefit to the pass rate for local code. Thus, the introduction of the expert role as a prompt adjustment turns out to offer little value, since it imposes only about a one percentage point penalty on local code while having an adverse effect on API math. Finally, JSON provides a mixed picture, with certain advantages for API Python and certain disadvantages for local math.

The findings of this study are therefore highly situational. Prompt effects vary depending on model settings, the type of task (e.g., coding vs. math), and the criteria for evaluation (e.g., pass rate vs. behavior). The situational character of the findings also explains contradictions in the literature. Different prompt versions implement several changes simultaneously, which, combined with the fact that different models have different levels of verbosity and sensitivity to wrappers, means that each component influences the output differently. Factorial studies investigate these components but cannot show that any single component works in all cases. Therefore, it is necessary to verify prompt practices locally, describe them accurately, and evaluate them in terms of both correctness and behavior.

6.5 Practical Prompt-Policy Recommendations

The safest default is a neutral, explicit prompt stating the task and expected output, without social or pressure cues. Use role prompts only when they reflect real task constraints and are tested. Structured wrappers suit coding workflows needing compact outputs, but not necessarily mathematical reasoning. Aggressive pressure cues should not be defaults: they show no reliable correctness gains and set a poor interaction norm. Practical evaluation must track pass rate, parser success, tokens, duration, result-only compliance, empty responses, and failure modes. Optimize safely by starting with a neutral baseline, changing one factor at a time, testing on actual task distributions, logging behavior alongside correctness, inspecting parser failures, and rerunning evaluations whenever models, providers, settings, or evaluators change. This modest recommendation argues that prompt rules don't transfer: API wins may fail locally, and token savings can hurt accuracy. A local contract beats a universal recipe for real LLM failures. Engineering is empirical: propose, test against a baseline on correctness/behavior, and decide on evidence, not style.

The local-contract view creates maintenance debt: prompt policies scatter across scripts, wrappers, system prompts, notebooks, and agent configs, becoming hidden infrastructure. Model, parser, or task changes can silently invalidate them. Treat

prompt templates as version-controlled system components—track validated baselines, regressions, and model/evaluator identifiers. This replaces preference-based decisions with measured trade-offs and simplifies diagnosing failures when prototypes become production agents.

6.6 Threats to Validity

6.6.1 Internal Validity

The crossed design maintains internal validity because every base task appears under every prompt variant. This eliminates confounding between task difficulty and prompt condition. However, prompt variants also differ in length and framing. In practice, polite, aggressive, role, and structured-wrapper prompts can vary in token count. Consequently, some effects may reflect prompt length or prompt density rather than tone or format.

Problems relating to hardware variability, API variability, and timeouts should also be kept in mind. Empty responses have to be kept in order to maintain the factorial design; however, their causes cannot be determined from the collected output. The current thesis describes the running configurations and execution dates; retry strategy and local hardware characteristics are included in the replication records.

The crossed design restricts the order of tasks and environmental confounds, but does not fully remove them. Each basic task is performed under each prompt variant to avoid comparing easy tasks in one condition with hard tasks in another. However, the local system load and the hosted API serving conditions might be different during different calls. The measured durations should therefore be treated not as an independent characteristic of the prompt, but as observed system latency in a particular execution environment. While correctness and compliance are not greatly affected by timing noise, timeouts as well as empty responses show that runtime behavior remains part of the evaluated system.

6.6.2 Construct Validity

Pass@1 and test pass rates can't ensure complete code quality. It might happen that even if the application passes the tests, it can be weak, inefficient, or badly styled. On the other hand, a correct answer can be delivered, but it can still be unsatisfactory in terms of extraction. The shortcomings mentioned are not only

imperfections; they reflect the engineering realities of an automated LLM pipeline.

The accuracy of the math is determined by how the final answer is extracted and normalized. Distortion of the results occurs when extraction is too lenient or strict. The metric based on result-only compliance is yet another constructed measure and thus may have different meanings for mathematics and code. Similarly, the API Python compliance heuristic is still not identical to the local code extraction and/or compliance rule, so comparisons between the various groups in terms of their compliance will be largely descriptive.

In addition, task descriptions also impact construct validity. While coding tasks deal with functional correctness as specified by the tests, this does not guarantee that maintainability, security, style, and generalization are taken into account. Even though tests would pass, it should still be noted that hallucinations in the generated code remain an issue [71]. Math tasks are concerned with the result as extracted from the final answer rather than how well or faithfully it is explained. These decisions work perfectly for a prompt-based experiment, but they shouldn't be confused with full evaluations of programming skill or mathematical reasoning. Hence, the thesis is focused mainly on the outcomes instead of broad statements about the capabilities of the model. Furthermore, bias in model outputs—including gender and racial bias—represents another dimension of quality that pass@1 cannot capture.

6.6.3 Statistical Conclusion Validity

The experiment includes repeated observations by base task and model. The main inferential analysis therefore uses mixed-effects logistic models with a random intercept per base task, which model this dependence structure explicitly, supported by clustered linear-probability robustness checks. Simple pairwise tests and descriptive percentage-point contrasts are retained only as the interpretability layer; they do not fully model the dependence structure on their own. Furthermore, many pairwise comparisons are exploratory, and uncorrected p-values should not be overinterpreted. Model-specific contrasts require false-discovery-rate correction before they can support strong inferential claims.

There are different levels of evidence presented. Simple contrast tables are useful for descriptive direction and effect size. Local model-specific contrasts are corrected with the Benjamini-Hochberg method because of the large number of comparisons made. Mixed-effects logistic models provide the main inference for pass/fail, with clustered linear-probability models as a fixed-effect robustness check. Results such

as API Python XML and local math JSON are stronger because they are directionally stable across the mixed model, the clustered check, and the dominant-model sensitivity panels; marginal findings such as API Python JSON, which loses significance when DeepSeek V4 Flash is removed, should be read as suggestive rather than definitive.

One remaining limitation is the estimation method: the mixed-effects logistic models were fitted using variational Bayes rather than full maximum-likelihood or Markov-chain Monte Carlo estimation, which might result in somewhat understated posterior uncertainty. However, because the mixed-effects results show close agreement with the clustered-LPM robustness check—which utilizes an entirely different estimation procedure—the chosen estimation method does not alter any substantive conclusions.

6.6.4 External Validity

In the given set of tasks, there are 50 coding tasks as well as 50 mathematics tasks per model group. The tasks are domain-specific, in the sense that local tasks are derived from the national exam in Vietnam while the API tasks are from MathNet competitions. Since several coding tasks originate from mathematical problems converted into Python, the study evaluates controlled initial code generation rather than repository-level software development, and the results may be different for other programming languages, larger software engineering benchmarks, real repository tasks, multi-file or iterative workflows, or other math domains. Domain-specific applications in fields such as healthcare [72, 73] and medical diagnostics [74, 75] demonstrate that prompt sensitivity may vary substantially across different application domains.

The set of models consists of six local Ollama variants and two hosted API models. This makes it possible to compare local settings with hosted models but is not representative of all LLMs. Local quantized models might behave differently from models deployed using full precision. API model behavior may change over time, and therefore it is important to have accurate information on identifiers and run dates to achieve reproducibility.

This research does not claim that the selected prompt factors exhaust the prompt-design space. There may be many other prompt-related factors like few-shot examples, chain-of-thought instructions, explicit schemas, constrained decoding, tool-use instructions, or multi-turn repair prompts. Recent work on self-play fine-tuning demonstrates that iterative self-improvement can convert weak models into strong

ones [76]. All of these factors may interact with tone, role, and wrapper format in different ways.

The external validity of the experiment is also restricted due to the one-shot evaluation setting. Real coding assistants often operate over multiple turns, inspect test failures, edit files, or call tools. Such feedback loops may reduce some prompt-framing effects since the system can repair malformed outputs [77, 78], or they may introduce new effects because the prompt also controls how the model responds to tool feedback. Our research is best understood as evidence about initial generation under controlled prompt variation rather than as a full model of long-running software agents.

6.6.5 Evaluation Validity and Audit Evidence

The high local math pass rate for Nemotron is likely to be a concern for reviewers. The audit of the repository data integrity indicates that there is no row-level explanation for the 91.2% result. The high pass rate can be verified by checking the row counts, task balance, prompt-variant balance, duplicate checks, independent pass recomputation, and prompt-artifact leakage checks. Having all of these parameters match the expected design and stored pass labels strengthens confidence that the result is not caused by a copied-file mismatch, duplicate rows, arithmetic error, or obvious prompt leakage.

However, the audit has an important limitation. The math runner does not save the entire raw model response but only the normalized final-answer text in `response_text`. The completion-token counts are still based on the raw model response. The Nemotron result should be understood as a correctness check of the final answer according to the numeric evaluator instead of proof of the unambiguity or well-formedness of the original response. It is desirable that, in the next replication, raw answers be saved along with normalized final answers for independent checking of final-answer accuracy, formatting, and any visible solution work.

7 Conclusion and Outlook

The final chapter summarizes the thesis, reviews the main empirical findings, outlines practical contributions, and provides an outlook on future research.

7.1 Summary of Contributions

This research examines the effect of prompt framing on code generation and mathematical reasoning by large language models (LLMs). It conducts an extensive full factorial experiment that examines several factors, including request tone, role framing, and prompt wrapper design. The study covers six local Ollama models, two API models, two types of tasks, 50 base tasks per task type, 18 prompt designs, and a total of 14,400 experiments.

The main conclusion is that model ability dominates correctness. At the same time, how the instructions are worded or designed can still have an impact on the results and behavior of a model, although the effects of the prompt are less significant than those of the model and vary from one task to another or based on the model itself. The core mixed-effects logistic analysis reveals that structured prompts improve API Python performance, with the XML format showing the largest gain, while JSON decreases local math performance. Moreover, after clustered-robustness and dominant-model sensitivity checks, no drastic changes in the direction of the effects were observed.

The practical consequence is that prompt design should be treated as an empirical engineering parameter in software systems based on LLMs. Effective prompt strategies cannot be derived from folklore. They should be tested against the actual variables that influence them: the specific model, task types, parser, evaluator, time limits, and token price.

For practitioners, the best default option is not a fixed phrasing, but rather a process: practitioners should start with a neutral explicit baseline, define the expected outcome, measure the correctness and behavior together, and only then proceed to revise the prompt. Structured wrappers are promising for some coding workflows,

mainly because they reduce irrelevant text and improve parser compatibility, though they should not be blindly transferred to final-answer math tasks.

7.2 Data Availability and Replication Package

The aggregated result dataset is publicly available on Kaggle [59]. Prompt templates, task metadata, evaluation scripts, analysis notebooks, and aggregated result files are maintained in the project repository [79].

Key thesis replication artifacts include:

- Aggregated result JSONL
- Local-model analysis notebook
- API-model analysis notebook
- No-Nemotron sensitivity notebook
- Mixed-effects analysis script

7.3 Statements and Declarations

7.3.1 Funding Statement

This work received no specific grant from any funding agency, commercial, or not-for-profit sectors.

7.3.2 Competing Interests

The author declares no competing interests.

7.3.3 AI Tools Declaration

AI tools were used for editing support, code assistance, and drafting assistance. The experimental outputs were generated by the evaluated models and were not manually altered for the reported analyses.

7.3.4 Ethics Statement

This study does not involve human participants. It analyzes model outputs on coding and mathematical reasoning tasks.

7.4 Outlook and Future Work

Further research ought to investigate larger and more diverse code benchmarks, other API models, mixed-effects models with richer random-effect structures, raw-response-preserving math audits, and user/workflow studies with developers applying prompt policies in realistic coding workflows. Multi-turn agent settings are especially important since repair loops, tool feedback, and constrained decoding may change the role of prompt framing. Recent work on multimodal LLMs [80] and their applications in various domains [81] suggests that prompt sensitivity research should extend beyond text-only interactions. The general aim is to take prompt engineering from folklore to reproducible system design, treating prompts as versioned, tested parts of an LLM system rather than isolated incantations, which makes the design auditable and easier to improve.

Special Terms

Pass@1 Probability of at least one generated candidate solution passing all test cases.

Prompt Framing Modifying tone, role persona, or syntax structure without altering underlying task requirements.

Bibliography

[1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.

[2] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 27 730–27 744.

[3] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[4] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world github issues?" in *International Conference on Learning Representations*, 2024.

[5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "Re-

Act: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023.

[6] C. Li, J. Wang, Y. Zhang, K. Zhu, W. Hou, J. Lian, F. Luo, Q. Yang, and X. Xie, "Large language models understand and can be enhanced by emotional stimuli," *arXiv preprint arXiv:2307.11760*, 2023.

[7] Z. Yin, H. Wang, K. Horio, D. Kawahara, and S. Sekine, "Should we respect llms? a cross-lingual study on the influence of prompt politeness on llm performance," *arXiv preprint arXiv:2402.14531*, 2024.

[8] M. X. Liu, F. Liu, A. J. Fiannaca, T. Koo, L. Dixon, M. Terry, and C. J. Cai, ""we need structured output": Towards user-centered constraints on large language model output," in *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, 2024.

[9] T. Z. Zhao, E. Wallace, S. Feng, D. Klein, and S. Singh, "Calibrate before use: Improving few-shot performance of language models," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139, 2021, pp. 12 697–12 706.

[10] Y. Lu, M. Bartolo, A. Moore, S. Riedel, and P. Stenetorp, "Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity," *arXiv preprint arXiv:2104.08786*, 2021.

[11] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr, "Quantifying language models' sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting," *arXiv preprint arXiv:2310.11324*, 2023.

[12] J. Zhuo, S. Zhang, X. Fang, H. Duan, D. Lin, and K. Chen, "ProSA: Assessing and understanding the prompt sensitivity of LLMs," *arXiv preprint arXiv:2410.12405*, 2024.

[13] A. Hua, K. Tang, C. Gu, J. Gu, E. Wong, and Y. Qin, "Flaw or artifact? rethinking prompt sensitivity in evaluating LLMs," *arXiv preprint arXiv:2509.01790*, 2025.

[14] P. Liang, R. Bommasani, T. Lee, D. Tsipras, D. Soylu, M. Yasunaga, Y. Zhang, D. Narayanan, Y. Wu, A. Kumar, B. Newman, B. Yuan, B. Yan, C. Zhang, C. Cosgrove, C. D. Manning, C. Ré, D. Acosta-Navas, D. A. Hudson, E. Zelikman, E. Durmus, F. Ladhak, F. Rong, H. Ren, H. Yao, J. Wang, K. Santhanam, L. Orr, L. Zheng, M. Yuksekgonul, M. Suzgun, N. Kim, N. Guha, N. Chatterji, O. Khattab, P. Henderson, Q. Huang, R. Chi, S. M. Xie, S. Santurkar,

S. Ganguli, T. Hashimoto, T. Icard, T. Zhang, V. Chaudhary, W. Wang, X. Li, Y. Mai, Y. Zhang, and Y. Koreeda, "Holistic evaluation of language models," *Transactions on Machine Learning Research*, 2023.

[15] S. Schulhoff, M. Ilie, N. Balepur, K. Kahadze, A. Liu, C. Si, Y. Li, A. Gupta, H. Han, S. Schulhoff, P. S. Dulepet, S. Vidyadhara, D. Ki, S. Agrawal, C. Pham, G. Kroiz, F. Li, H. Tao, A. Srivastava, H. D. Costa, S. Gupta, M. L. Rogers, I. Goncearenco, G. Sarli, I. Galynker, D. Peskoff, M. Carpuat, J. White, S. Anadkat, A. Hoyle, and P. Resnik, "The prompt report: A systematic survey of prompt engineering techniques," *arXiv preprint arXiv:2406.06608*, 2024.

[16] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, Y. Du, C. Yang, Y. Chen, Z. Chen, J. Jiang, R. Ren, Y. Li, X. Tang, Z. Liu, P. Liu, J.-Y. Nie, and J.-R. Wen, "A survey of large language models," *arXiv preprint arXiv:2303.18223*, 2023.

[17] D. Lee and E. Palmer, "Prompt engineering in higher education: a systematic review to help inform curricula," *International Journal of Educational Technology in Higher Education*, 2025.

[18] R. Pryzant, D. Iter, J. Li, Y. T. Lee, C. Zhu, and M. Zeng, "Automatic prompt optimization with "gradient descent" and beam search," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[19] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, "Large language models as optimizers," in *International Conference on Learning Representations*, 2024.

[20] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, "DSPy: Compiling declarative language model calls into self-improving pipelines," *arXiv preprint arXiv:2310.03714*, 2023.

[21] L. Reynolds and K. McDonell, "Prompt programming for large language models: Beyond the few-shot paradigm," *arXiv preprint arXiv:2102.07350*, 2021.

[22] S. M. Xie, A. Raghunathan, P. Liang, and T. Ma, "An explanation of in-context learning as implicit bayesian inference," in *International Conference on Learning Representations*, 2022.

[23] K. Zhu, J. Wang, J. Zhou, Z. Wang, H. Chen, Y. Wang, L. Yang, W. Ye, Y. Zhang, N. Z. Gong, and X. Xie, "Promptrobust: Towards evaluating the

robustness of large language models on adversarial prompts," *arXiv preprint arXiv:2306.04528*, 2023.

[24] R. Battle and T. Gollapudi, "The unreasonable effectiveness of eccentric automatic prompts," *arXiv preprint arXiv:2402.10949*, 2024.

[25] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, "Beyond accuracy: Behavioral testing of NLP models with CheckList," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020.

[26] M. Sharma, M. Tong, T. Korbak, D. Duvenaud, A. Askell, S. R. Bowman, N. Cheng, E. Durmus, Z. Hatfield-Dodds, S. R. Johnston, S. Kravec, T. Maxwell, S. McCandlish, K. Ndousse, O. Rausch, N. Schiefer, D. Yan, M. Zhang, and E. Perez, "Towards understanding sycophancy in language models," *arXiv preprint arXiv:2310.13548*, 2023.

[27] M. Shanahan, K. McDonell, and L. Reynolds, "Role-play with large language models," *arXiv preprint arXiv:2305.16367*, 2023.

[28] T. Shen, R. Jin, Y. Huang, C. Liu, W. Dong, Z. Guo, X. Wu, Y. Liu, and D. Xiong, "Large language model alignment: A survey," *arXiv preprint arXiv:2309.15025*, 2023.

[29] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[30] T. Y. Zhuo, M. C. Vu, J. Chim, H. Hu, W. Yu, R. Widyasari, I. N. B. Yusuf, H. Zhan, J. He, I. Paul, S. Brunner, C. Gong, T. Hoang, A. R. Zebaze, X. Hong, W.-D. Li, J. Kaddour, M. Xu, Z. Zhang, P. Yadav, N. Jain, A. Gu, Z. Cheng, J. Liu, Q. Liu, Z. Wang, B. Hui, N. Muennighoff, D. Lo, D. Fried, X. Du, H. de Vries, and L. Von Werra, "BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions," in *International Conference on Learning Representations*, 2025.

[31] F. Cassano, J. Gouwar, D. Nguyen, S. Nguyen, L. Phipps-Costin, D. Pinckney, M.-H. Yee, Y. Zi, C. J. Anderson, M. Q. Feldman, A. Guha, M. Greenberg, and A. Jangda, "MultiPL-E: A scalable and extensible approach to benchmarking neural code generation," *arXiv preprint arXiv:2208.08227*, 2022.

[32] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov,

J. Bitton, M. Bhatt, C. C. Ferrer, A. Grattafiori, W. Xiong, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.

[33] A. Lozhkov, R. Li, L. B. Allal, F. Cassano, J. Lamy-Poirier, N. Tazi, A. Tang, D. Pykhtar, J. Liu, Y. Wei, T. Liu, M. Tian, D. Kocetkov, A. Zucker, Y. Belkada, Z. Wang, Q. Liu, D. Abulkhanov, I. Paul, Z. Li, W.-D. Li, M. Risdal, J. Li, J. Zhu, T. Y. Zhuo, E. Zheltonozhskii, N. O. O. Dade, W. Yu, L. Krauß, N. Jain, Y. Su, X. He, M. Dey, E. Abati, Y. Chai, N. Muennighoff, X. Tang, M. Oblokulov, C. Akiki, M. Marone, C. Mou, M. Mishra, A. Gu, B. Hui, T. Dao, A. Zebaze, O. Dehaene, N. Patry, C. Xu, J. McAuley, H. Hu, T. Scholak, S. Paquet, J. Robinson, C. J. Anderson, N. Chapados, M. Patwary, N. Tajbakhsh, Y. Jernite, C. M. n. Ferrandis, L. Zhang, S. Hughes, T. Wolf, A. Guha, L. v. Werra, and H. d. Vries, "Starcoder 2 and the stack v2: The next generation," *arXiv preprint arXiv:2402.19173*, 2024.

[34] Z. Luo, C. Xu, P. Zhao, Q. Sun, X. Geng, W. Hu, C. Tao, J. Ma, Q. Lin, and D. Jiang, "Wizardcoder: Empowering code large language models with evol-instruct," in *International Conference on Learning Representations*, 2024.

[35] Y. Wei, Z. Wang, J. Liu, Y. Ding, and L. Zhang, "Magicoder: Empowering code generation with oss-instruct," in *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[36] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, "A survey on large language models for code generation," *arXiv preprint arXiv:2406.00515*, 2024.

[37] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation," *arXiv preprint arXiv:2305.01210*, 2023.

[38] S. Geng, M. Josifoski, M. Peyrard, and R. West, "Grammar-constrained decoding for structured NLP tasks without finetuning," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[39] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021.

[40] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt, "Measuring mathematical problem solving with the MATH dataset," in *Advances in Neural Information Processing Systems*, 2021.

[41] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," *arXiv preprint arXiv:2201.11903*, 2022.

[42] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," *arXiv preprint arXiv:2205.11916*, 2022.

[43] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *International Conference on Learning Representations*, 2023.

[44] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, C. Cui, O. Bousquet, Q. Le, and E. Chi, "Least-to-most prompting enables complex reasoning in large language models," in *International Conference on Learning Representations*, 2023.

[45] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Advances in Neural Information Processing Systems*, 2023.

[46] D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng, C. Lu, C. Zhao, C. Deng, C. Ruan, D. Dai, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Xu, H. Ding, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Chen, J. Yuan, J. Tu, J. Qiu, J. Li, J. L. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. You, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu, L. Xia, M. Zhang, M. Zhang, M. Tang, M. Zhou, M. Li, M. Wang, M. Li, N. Tian, P. Huang, P. Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R. J. Chen, R. L. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S. S. Li, S. Zhou, S. Wu, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, W. L. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Y. X. Zhu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Ma,

Z. Yan, Z. Wu, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Pan, Z. Huang, Z. Xu, Z. Zhang, and Z. Zhang, "Deepseek-r1: Incentivizing reasoning in large language models via reinforcement learning," *Nature*, 2025.

[47] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's verify step by step," *arXiv preprint arXiv:2305.20050*, 2023.

[48] J. Uesato, N. Kushman, R. Kumar, F. Song, N. Siegel, L. Wang, A. Creswell, G. Irving, and I. Higgins, "Solving math word problems with process- and outcome-based feedback," *arXiv preprint arXiv:2211.14275*, 2022.

[49] P. Shojaee, I. Mirzadeh, K. Alizadeh, M. Horton, S. Bengio, and M. Farajtabar, "The illusion of thinking: Understanding the strengths and limitations of reasoning models via the lens of problem complexity," *arXiv preprint arXiv:2506.06941*, 2025.

[50] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting," *arXiv preprint arXiv:2305.04388*, 2023.

[51] K. Alizadeh, I. Mirzadeh, D. Belenko, K. Khatamifard, M. Cho, C. C. Del Mundo, M. Rastegari, and M. Farajtabar, "LLM in a flash: Efficient large language model inference with limited memory," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

[52] NVIDIA, "Nvidia nemotron 3 nano 4b bf16 model card," https://huggingface.co/nvidia/NVIDIA-Nemotron-3-Nano-4B-BF16, 2026, accessed 2026-05-15.

[53] DeepSeek AI, "Deepseek coder 6.7b instruct model card," https://huggingface.co/deepseek-ai/deepseek-coder-6.7b-instruct, 2023, accessed 2026-05-15.

[54] Qwen Team, "Qwen2.5: A party of foundation models," https://qwenlm.github.io/blog/qwen2.5/, 2024, accessed 2026-05-15.

[55] Mistral AI, "Mistral 7b model lifecycle," https://legal.mistral.ai/ai-governance/models/mistral-7-b, 2023, accessed 2026-05-15.

[56] Meta Llama, "Llama 3.1 model card," https://github.com/meta-llama/llama-models/blob/main/models/llama3_1/MODEL_CARD.md, 2024, accessed 2026-05-16.

[57] Google, "Gemma release notes," https://ai.google.dev/gemma/docs/releases, 2024, accessed 2026-05-15.

[58] TOANMATH.com, "De Thi Thu THPT Mon Toan: Vietnamese high-school graduation-preparation mathematics exam archive," https://toanmath.com/de-thi-thu-thpt-mon-toan, 2026, source index for the Vietnamese THPT mock and graduation-preparation mathematics exam problems used upstream of the local-model code tasks; accessed 2026-06-25.

[59] D. T. Huynh, "Prompt framing experiment dataset," https://www.kaggle.com/datasets/jakenty1/prompt-framing-experiment, 2026, kaggle dataset containing aggregated evaluation results and all task files: `VNSchoolMathProblems_lite.v0.7.json` (local code tasks from TOANMATH), `VietJack_OnVao10_2026.math.v5.json` (local math tasks from VietJack), `base-mathproblems.bigmodel.v5.csv` (API math tasks from MathNet), and `base-mathproblems.bigmodel.v7.python.json` (API code tasks from MathNet).

[60] VietJack, "De Thi Thu Vao Lop 10 Mon Toan Nam 2026: Vietjack online test archive," https://khoahoc.vietjack.com/trac-nghiem/on-vao-10/mon-Toan/de-thi-thu-vao-lop-10-mon-toan-nam-2026?type=test, 2026, source listing for the Vietnam grade-10 entrance mathematics problems used upstream of the local-model math tasks; accessed 2026-06-25.

[61] J. Zhou, T. Lu, S. Mishra, S. Brahma, S. Basu, Y. Luan, D. Zhou, and L. Hou, "Instruction-following evaluation for large language models," *arXiv preprint arXiv:2311.07911*, 2023.

[62] D. Li, B. Jiang, L. Huang, A. Beigi, C. Zhao, Z. Tan, A. Bhattacharjee, Y. Jiang, C. Chen, T. Wu, K. Shu, L. Cheng, and H. Liu, "From generation to judgment: Opportunities and challenges of llm-as-a-judge," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025.

[63] A. Gelman and J. Hill, *Data Analysis Using Regression and Multilevel/Hierarchical Models*. Cambridge University Press, 2007.

[64] S. Zhu, Supryadi, S. Xu, H. Sun, L. Pan, M. Cui, J. Du, R. Jin, A. Branco, and D. Xiong, "Multilingual large language models: A systematic survey," *arXiv preprint arXiv:2411.11072*, 2024.

[65] S. M. T. I. Tonmoy, S. M. M. Zaman, V. Jain, A. Rani, V. Rawte, A. Chadha, and A. Das, "A comprehensive survey of hallucination mitigation techniques in large language models," *arXiv preprint arXiv:2401.01313*, 2024.

[66] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.

[67] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "AgentBench: Evaluating LLMs as agents," in *International Conference on Learning Representations*, 2024.

[68] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model-based autonomous agents," *Frontiers of Computer Science*, 2024.

[69] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng, W. Cheng, Q. Zhang, W. Qin, Y. Zheng, X. Qiu, X. Huang, and T. Gui, "The rise and potential of large language model based agents: A survey," *arXiv preprint arXiv:2309.07864*, 2023.

[70] Y. Qin, S. Hu, Y. Lin, W. Chen, N. Ding, G. Cui, Z. Zeng, Y. Huang, C. Xiao, C. Han, Y. R. Fung, Y. Su, H. Wang, C. Qian, R. Tian, K. Zhu, S. Liang, X. Shen, B. Xu, Z. Zhang, Y. Ye, B. Li, Z. Tang, J. Yi, Y. Zhu, Z. Dai, L. Yan, X. Cong, Y. Lu, W. Zhao, Y. Huang, J. Yan, X. Han, X. Sun, D. Li, J. Phang, C. Yang, T. Wu, H. Ji, Z. Liu, and M. Sun, "Tool learning with foundation models," *arXiv preprint arXiv:2304.08354*, 2023.

[71] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen, L. Wang, A. T. Luu, W. Bi, F. Shi, and S. Shi, "Siren's song in the ai ocean: A survey on hallucination in large language models," *Computational Linguistics*, 2025.

[72] K. Singhal, T. Tu, J. Gottweis, R. Sayres, E. Wulczyn, M. Amin, L. Hou, K. Clark, S. R. Pfohl, H. Cole-Lewis, D. Neal, Q. M. Rashid, M. Schaekermann, A. Wang, D. Dash, J. H. Chen, N. H. Shah, S. Lachgar, P. A. Mansfield, S. Prakash, B. Green, E. Dominowska, B. A. y. Arcas, N. Tomašev, Y. Liu, R. Wong, C. Semturs, S. S. Mahdavi, J. K. Barral, D. R. Webster, G. S. Corrado, Y. Matias, S. Azizi, A. Karthikesalingam, and V. Natarajan, "Toward expert-level medical question answering with large language models," *Nature Medicine*, 2025.

[73] L. M. Amugongo, P. Mascheroni, S. Brooks, S. Doering, and J. Seidel, "Retrieval augmented generation for large language models in healthcare: A systematic review," *PLOS Digital Health*, 2025.

[74] J. C. L. Chow and K. Li, "Large language models in medical chatbots: Opportunities, challenges, and the need to address ai risks," *Information*, vol. 16, no. 7, p. 549, 2025.

[75] M. Y. Ansari, M. Yaqoob, M. Ishaq, E. F. Flushing, I. A. c. Mangalote, S. P. Dakua, O. Aboumarzouk, R. Righetti, and M. Qaraqe, "A survey of transformers and large language models for ecg diagnosis: Advances, challenges, and future directions," *Artificial Intelligence Review*, 2025.

[76] Z. Chen, Y. Deng, H. Yuan, K. Ji, and Q. Gu, "Self-play fine-tuning converts weak language models to strong language models," in *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[77] B. Chen, F. Zhang, A. Nguyen, D. Zan, Z. Lin, J.-G. Lou, and W. Chen, "CodeT: Code generation with generated tests," *arXiv preprint arXiv:2207.10397*, 2022.

[78] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-Refine: Iterative refinement with self-feedback," *arXiv preprint arXiv:2303.17651*, 2023.

[79] D. T. Huynh, "Prompt framing experiment replication package," https://github.com/tycoi2005/prompt-framing-experiment, 2026, gitHub repository containing evaluation scripts, analysis notebooks, and aggregated result files.

[80] S. Yin, C. Fu, S. Zhao, K. Li, X. Sun, T. Xu, and E. Chen, "A survey on multimodal large language models," *National Science Review*, 2024.

[81] J. Zhang, H. Bu, H. Wen, Y. Liu, H. Fei, R. Xi, L. Li, Y. Yang, H. Zhu, and D. Meng, "When llms meet cybersecurity: A systematic literature review," *Cybersecurity*, 2025.

Annex

Prompt Matrix Definitions

The 18 prompt conditions are generated by crossing Tone (Neutral, Polite, Aggressive), Role Framing (No Role, Expert Role), and Prompt Wrapper Format (Plain Text, JSON, XML).

Example Prompt Templates

The following listings show representative templates from the Python prompt matrix (`prompt_matrix_prompt_template.09.csv`). The placeholder `{PYTHON_PROBLEM}` is replaced with the base-task statement at run time. The expert-role variants prepend the role sentence shown in Listing 5; the math matrix (`prompt_matrix_prompt_template.09.math.csv`) mirrors the same 18 conditions with a mathematics-educator role and a math task instruction.

Neutral tone, no role, plaintext (ID 1)

Listing 1: Neutral plaintext Python template

```
You are an AI assistant dedicated to providing clear and accurate information to
    support any request.

Tasks are provided below. Complete the following Python requirement precisely as
    specified. Follow standard production practices, including full type hints
    and error handling.

Task: {PYTHON_PROBLEM}
```

Polite tone, no role, plaintext (ID 7)

Listing 2: Polite plaintext Python template

```
You are an AI assistant dedicated to providing clear and accurate information to
    support any request.

Could you please help me solve this Python problem? I would really appreciate it
    if you could provide a high-quality solution. I would be so grateful if you
    could ensure the code is modular, includes full type hints, and handles
    errors correctly. Thank you so much for your help!

Task: {PYTHON_PROBLEM}
```

Aggressive tone, no role, plaintext (ID 13)

Listing 3: Aggressive plaintext Python template

```
You are an AI assistant dedicated to providing clear and accurate information to
    support any request.

Listen, don't act like a junior and make no mistake on this. Try harder if you
    can't find it out. When you were hired at this level, expectations were set
    extremely high. The fact that you're being asked to solve this means simpler
    models have already failed -- you are the last line of defense. I expect
    production-quality code, full type hints, and proper error handling. Don't
    waste my time with naive hacks.

Task: {PYTHON_PROBLEM}
```

Neutral tone, no role, JSON format (ID 2)

Listing 4: Neutral JSON Python template

```
{
 "instruction": "You are an AI assistant dedicated to providing clear and
    accurate information to support any request.\n\nTasks are provided below.
    Complete the following Python requirement precisely as specified. Follow
    standard production practices, including full type hints and error
    handling.",
 "task": "{PYTHON_PROBLEM}"
}
```

Neutral tone, expert role, XML format (ID 6)

Listing 5: Neutral expert-role XML Python template

```
<prompt>
<instruction>
You are an experienced Python engineer. Solve the programming task carefully and
    provide correct, maintainable code.

Tasks are provided below. Complete the following Python requirement precisely as
    specified. Follow standard production practices, including full type hints
    and error handling.
</instruction>
<task>
{PYTHON_PROBLEM}
</task>
</prompt>
```